

ГЛАВА 19

СПЕЦИАЛИЗИРОВАННЫЕ ИНСТРУМЕНТЫ И ТЕХНОЛОГИИ

В этой главе...

19.1. Контроль распределения памяти	1023
19.2. Идентификация типов времени выполнения	1029
19.3. Перечисления	1038
19.4. Указатель на член класса	1042
19.5. Вложенные классы	1052
19.6. Класс объединения, экономящий место	1056
19.7. Локальные классы	1063
19.8. Возможности, снижающие переносимость	1065
Резюме	1072
Термины	1073

В первых трех частях этой книги обсуждались аспекты языка C++, используемые практически всеми программистами C++. Кроме того, язык C++ предоставляет некоторые специализированные средства, которые большинство программистов используют крайне редко или не используют вообще.

Язык C++ предназначен для создания самых разнообразных приложений. В результате он обладает средствами, ненужными для одних приложений и иногда используемыми в других. В этой главе рассматриваются довольно редко используемые средства языка C++.

19.1. Контроль распределения памяти

Некоторые приложения нуждаются в специализированном распределении памяти, которое не могут обеспечить стандартные средства управления памятью. Разработчики таких приложений вынуждены вникать в подробности резервирования памяти, например, применения оператора `new` для помещения объекта в специфические виды памяти. Для этого они могут перегрузить операторы `new` и `delete` так, чтобы самостоятельно контролировать распределение памяти.

19.1.1. Перегрузка операторов `new` и `delete`

Хотя говорят, что можно “перезагрузить операторы `new` и `delete`”, перегрузка этих операторов весьма отличается от способа перегрузки других операторов. Чтобы понять, как их можно перезагрузить, следует сначала узнать больше о том, как работают выражения `new` и `delete`.

Выражение `new` используется так:

```
// выражение new
string *sp = new string("a value"); // зарезервировать и инициализировать
                                   // строку
string *arr = new string[10];       // зарезервировать десять строк,
                                   // инициализированных значением по
                                   // умолчанию
```

Фактически здесь три этапа: сначала выражение вызывает библиотечную функцию `operator new()` (или `operator new[]()`). Эта функция резервирует не типизированную область памяти достаточного размера для содержания объекта (или массива объектов) определенного типа. Затем компилятор запускает соответствующий конструктор, чтобы создать объект (объекты) из переданных инициализаторов. И наконец, возвращается указатель на вновь зарезервированный и созданный объект.

Выражение `delete` применяется для удаления динамически созданного объекта:

```
delete sp;      // удалить *sp и освободить память, на которую указывает sp
delete [] arr;  // удалить элементы массива и освободить память
```

Здесь два этапа: сначала для объекта, на который указывает указатель `sp`, или для элементов массива, на который указывает имя `arr`, выполняется соответствующий деструктор. Затем компилятор освобождает память, вызвав библиотечную функцию `operator delete()` или `operator delete[]()` соответственно.

Приложения, которые собираются самостоятельно контролировать распределение памяти, определяют собственные версии функций `operator new()` и `operator delete()`. Даже при том, что библиотека содержит определения этих функций, вполне можно определить их собственные версии, и компилятор не пожалуется на двойное определение. Вместо этого компилятор использует пользовательскую версию, а не определенную библиотекой.



ВНИМАНИЕ

При определении глобальных функций `operator new()` и `operator delete()` вся ответственность за динамическое распределение памяти ложится на разработчика. Эти функции *должны* быть корректны, так как являются жизненно важной частью всей программы.

Функции `operator new()` и `operator delete()` можно определить в глобальной области видимости или как функции-члены. Когда компилятор встреча-

ет выражение `new` или `delete`, он ищет соответствующую вызову функцию оператора. Если резервируемый (освобождаемый) объект имеет тип класса, то компилятор ищет сначала в пределах класса, включая все его базовые классы. Если у класса есть функции-члены `operator new()` и `operator delete()`, эти функции и используются в выражении `new` или `delete`. В противном случае компилятор ищет соответствующую функцию в глобальной области видимости. Если компилятор находит пользовательскую версию функции, он ее и использует для выполнения выражения `new` или `delete`. В противном случае используется версия из стандартной библиотеки.

Чтобы заставить выражение `new` или `delete` обойти функцию, предоставленную классом, и использовать таковую из глобальной области видимости, можно использовать оператор области видимости. Например, выражение `::new` имеет в виду функцию `operator new()` только из глобальной области видимости. Аналогично для выражения `::delete`.

Интерфейс функций `operator new()` и `operator delete()`

Библиотека определяет восемь перегруженных версий функций `operator new()` и `operator delete()`. Первые четыре версии оператора `new` способны передавать исключение `bad_alloc`. Следующие четыре версии оператора `new` не передают исключений:

```
// версии, способные передавать исключения
void *operator new(size_t);           // резервирует объект
void *operator new[](size_t);        // резервирует массив
void *operator delete(void*) noexcept; // освобождает объект
void *operator delete[](void*) noexcept; // освобождает массив
```

```
// версии, обещающие не передавать исключений; см. р. 12.1.2 (стр. 586)
void *operator new(size_t, nothrow_t&) noexcept;
void *operator new[](size_t, nothrow_t&) noexcept;
void *operator delete(void*, nothrow_t&) noexcept;
void *operator delete[](void*, nothrow_t&) noexcept;
```

Тип `nothrow_t` является структурой, определенной в заголовке `new`. У этого типа нет никаких членов. Заголовок `new` определяет также константный объект `nothrow`, который пользователи могут передавать как сигнал, что необходима версия оператора `new`, не передающего исключения (см. раздел 12.1.2, стр. 586). Будучи деструктором, функция `operator delete()` не должна передавать исключения (см. раздел 18.1.1, стр. 968). При перегрузке этих операторов следует определить, будут ли они передавать исключения. Для этого используется спецификатор исключения `noexcept` (см. раздел 18.1.4, стр. 974).

Приложение может определить свою собственную версию любой из этих функций. Если это так, то следует определить эти функции в глобальной области видимости или как функцию-член класса. Когда эти функции операторов

определены как члены класса, они неявно являются статическими (см. раздел 7.6, стр. 390). Нет никакой необходимости объявлять их статическими явно, хотя сделать это вполне допустимо. Функции-члены операторов `new` и `delete` должны быть статическими, поскольку они используются до создания объекта (`operator new`) или после его удаления (`operator delete`). Поэтому у них нет никаких переменных-членов, которыми они могли бы манипулировать.

У функций `operator new()` и `operator new[]()` должен быть тип возвращаемого значения `void*`, а их первый параметр должен иметь тип `size_t`. У этого параметра не может быть аргумента по умолчанию. Функция `operator new()` используется при резервировании объекта; функция `operator new[]()` вызывается при резервировании массива. Когда компилятор вызывает функцию `operator new()`, он инициализирует параметр типа `size_t` количеством байтов, необходимых для содержания объекта заданного типа; при вызове функции `operator new[]()` передается количество байтов, необходимых для хранения массива заданного количества элементов.

При определении собственной версии функции `operator new()` можно определить дополнительные параметры. Чтобы использующие такие функции выражения `new` могли передать аргументы этим дополнительным параметрам, следует применять размещающую форму оператора `new` (см. раздел 12.1.2, стр. 586). Хотя обычно вполне можно определить собственную версию функции `operator new()`, чтобы получить необходимый набор параметров, нельзя определить эту функцию в следующей форме:

```
void *operator new(size_t, void*); // эта версия не может быть  
                                // переопределена
```

Данная конкретная форма зарезервирована для использования библиотекой и не может быть переопределена.

У функций `operator delete()` и `operator delete[]()` должен быть тип возвращаемого значения `void` и первый параметр типа `void*`. Выполнение выражения `delete` вызывает соответствующую функцию оператора и инициализирует ее параметр типа `void*` указателем на область памяти, подлежащую освобождению.

Когда функции `operator delete()` и `operator delete[]()` определяются как члены класса, у них может быть второй параметр типа `size_t`. Этот дополнительный параметр инициализируется размером (в байтах) объекта, заданного первым параметром. Параметр типа `size_t` используется при удалении объектов, являющихся частью иерархии наследования. Если у базового класса есть виртуальный деструктор (см. раздел 15.7.1, стр. 785), то передаваемый функции `operator delete()` размер зависит от динамического типа объекта, на который указывает удаляемый указатель. Кроме того, выполняемая версия функции `operator delete()` также будет зависеть от динамического типа объекта.

ТЕРМИНОЛОГИЯ. ВЫРАЖЕНИЕ `new` ИЛИ ФУНКЦИЯ `operator new()`

Имена библиотечных функций `operator new()` и `operator delete()` могут ввести в заблуждение. В отличие от других функций операторов (таких как `operator=`), эти функции не перегружают операторы `new` и `delete`. Фактически переопределить поведение операторов `new` и `delete` нельзя.

В процессе выполнения оператор `new` вызывает функцию `operator new()`, чтобы зарезервировать область памяти, в которой он затем создает объект. Оператор `delete` удаляет объект, а затем вызывает функцию `operator delete()`, чтобы освободить использованную объектом память.

Функции `malloc()` и `free()`

Если определяются собственные глобальные функции `operator new()` и `operator delete()`, они должны резервировать и освобождать память так или иначе. Даже если эти функции определяются для использования специализированной системы резервирования памяти, может иметь смысл (для проверки) иметь способность резервировать память тем же способом, что и обычная реализация.

В этом случае можно использовать функции `malloc()` и `free()`, унаследованные языком C++ от языка C. Они определяются в заголовке `cstdlib`.

Функция `malloc()` получает параметр типа `size_t`, задающий количество резервируемых байтов. Она возвращает указатель на зарезервированную область памяти или значение 0, если зарезервировать память не удалось. Функция `free()` получает параметр типа `void*`, являющийся копией указателя, возвращенного функцией `malloc()`, и возвращает занятую память операционной системе. Вызов `free(0)` не делает ничего.

Вот простейший код функций `operator new()` и `operator delete()`:

```
void *operator new(size_t size) {
    if (void *mem = malloc(size))
        return mem;
    else
        throw bad_alloc();
}
void operator delete(void *mem) noexcept { free(mem); }
```

Для других версий функции `operator new()` и `operator delete()` код аналогичен.

Упражнения раздела 19.1.1

Упражнение 19.1. Напишите собственную версию функции `operator new(size_t)`, используя функцию `malloc()`, и версию функции `operator delete(void*)`, используя функцию `free()`.

Упражнение 19.2. По умолчанию класс `allocator` использует функцию `operator new()` для резервирования места и функцию `operator delete()` для ее освобождения. Перекомпилируйте и повторно запустите программу `StrVec` (см. раздел 13.5, стр. 666), используя собственные версии функций из предыдущего упражнения.

19.1.2. Размещающий оператор `new`

Хотя функции `operator new()` и `operator delete()` предназначены для использования выражениями `new`, они являются обычными библиотечными функциями. Поэтому обычный код вполне может вызвать их непосредственно.

В прежних версиях языка (до того, как класс `allocator` (см. раздел 12.2.2, стр. 613) стал частью библиотеки), когда необходимо было отделить резервирование от инициализации, использовались функции `operator new()` и `operator delete()`. Эти функции ведут себя аналогично функциям-членам `allocate()` и `deallocate()` класса `allocator` — резервируют и освобождают память, но не создают и не удаляют объекты.

В отличие от класса `allocator`, нет функции `construct()`, позволяющей создавать объекты в памяти, зарезервированной функцией `operator new()`. Вместо этого для создания объекта используется *размещающий оператор* `new` (`placement new`) (см. раздел 12.1.2, стр. 586). Как уже упоминалось, эта форма оператора `new` предоставляет дополнительную информацию функции резервирования. Размещающий оператор `new` можно использовать для передачи адреса области. Тогда выражения размещающего оператора `new` будут иметь следующую форму:

```
new (адрес_области) тип
new (адрес_области) тип (инициализаторы)
new (адрес_области) тип [размер]
new (адрес_области) тип [размер] { список инициализации }
```

где *адрес_области* является указателем, а *инициализаторы* представляют собой разделяемый запятыми список инициализаторов (возможно, пустой), используемый для создания вновь зарезервированного объекта.

Будучи вызванным с адресом, но без других аргументов, размещающий оператор `new` использует вызов `operator new(size_t, void*)` для “резервирования” памяти. Эта версия функции `operator new()` не допускает перепределения (см. раздел 19.1.1, стр. 1026). Она *не резервирует* память, а просто возвращает свой аргумент указателя. Затем обычное выражение `new` заканчивает свою работу инициализацией объекта по данному адресу. В действительности размещающий оператор `new` позволяет создать объект в заданной адресом предварительно зарезервированной области памяти.



При передаче одного аргумента, являющегося указателем, выражение размещающего оператора `new` создает объект, но не резервирует память.

Хотя существует несколько способов использования размещающего оператора `new`, он похож на функцию-член `construct()` класса `allocator`, но с одним важным отличием. Передаваемый функции `construct()` указатель должен указывать на область, зарезервированную тем же объектом класса `allocator`. Указатель, передаваемый размещающему оператору `new`, не обязан указывать на область памяти, зарезервированной функцией `operator new()`. Как будет продемонстрировано в разделе 19.6 (стр. 1061), переданный выражению размещающего оператора `new` указатель даже не обязан указывать на динамическую память.

Явный вызов деструктора

Подобно тому, как размещающий оператор `new` является низкоуровневой альтернативой функции-члену `allocate()` класса `allocator`, явный вызов деструктора аналогичен вызову функции `destroy()`.

Вызов деструктора происходит таким же образом, как и любой другой функции-члена объекта: через указатель или ссылку на объект:

```
string *sp = new string("a value"); // резервирует и инициализирует
                                     // строку
sp->~string();
```

Здесь деструктор вызывается непосредственно. Для получения объекта, на который указывает указатель `sp`, используется оператор стрелки. Затем происходит вызов деструктора, имя которого совпадает с именем типа, но с предваряющим знаком тильды (`~`).

Подобно вызову функции `destroy()`, вызов деструктора освобождает заданный объект, но не освобождает область, в которой располагается этот объект. При желании эту область можно использовать многократно.



Вызов деструктора удаляет объект, но не освобождает память.

19.2. Идентификация типов времени выполнения

Идентификацию типов времени выполнения (run-time type identification — RTTI) обеспечивают два оператора.

- Оператор `typeid`, возвращающий фактический тип заданного выражения.
- Оператор `dynamic_cast`, безопасно преобразующий указатель или ссылку на базовый тип в указатель или ссылку на производный.

Будучи примененными к указателям или ссылкам на тип с виртуальными функциями, эти операторы используют динамический тип (см. раздел 15.2.3, стр. 759) объекта, с которым связан указатель или ссылка.

Эти операторы полезны в случае, когда в производном классе имеется функция, которую необходимо выполнить через указатель или ссылку на объект базового класса, и эту функцию невозможно сделать виртуальной. Обычно по возможности лучше использовать виртуальные функции. Когда применяется виртуальная функция, компилятор автоматически выбирает правильную функцию согласно динамическому типу объекта.

Но определить виртуальную функцию не всегда возможно. В таком случае может пригодиться один из операторов RTTI. С другой стороны, эти операторы более склонны к ошибкам, чем виртуальные функции-члены: разработчик должен *знать*, к какому типу следует привести объект, и обеспечить проверку успешности приведения.



ВНИМАНИЕ

Динамическое приведение следует использовать осторожно. При каждой возможности желательно создавать и использовать виртуальные функции, а не прибегать к непосредственному управлению типами.

19.2.1. Оператор `dynamic_cast`

Оператор `dynamic_cast` имеет следующую форму:

```
dynamic_cast<тип*>(e)
dynamic_cast<тип&>(e)
dynamic_cast<тип&&>(e)
```

где *тип* должен быть типом класса, у которого (обычно) есть виртуальные функции. В первом случае *e* — допустимый указатель (см. раздел 2.3.2, стр. 87); во втором — l-значение, а в третьем — не должен быть l-значением.

Во всех случаях тип указателя *e* должен быть либо типом класса, открыто унаследованным от *типа* назначения, либо открытым базовым классом *типа* назначения, либо самим *типом* назначения. Если указатель *e* будет одним из этих типов, то приведение окажется успешным. В противном случае приведение закончится ошибкой. При неудаче приведения к типу указателя оператор `dynamic_cast` возвращает 0. При неудаче приведения к типу ссылки он передает исключение типа `bad_cast`.

Приведение `dynamic_cast` для типа указателя

Для примера рассмотрим класс `Base`, обладающий по крайней мере одной виртуальной функцией-членом, и класс `Derived`, открыто унаследованный от класса `Base`. Если имеется указатель `bp` на класс `Base`, то во время выполнения можно привести его к указателю на тип `Derived` следующим образом:

```
if (Derived *dp = dynamic_cast<Derived*>(bp))
{
    // использование объекта Derived, на который указывает dp
} else { // bp указывает на объект Base
    // использование объекта Base, на который указывает dp
}
```

Если `bp` указывает на объект класса `Derived`, то приведение инициализирует указатель `dp` так, чтобы он указывал на объект класса `Derived`, на который указывает указатель `bp`. В данном случае для кода в операторе `if` вполне безопасно использовать функции класса `Derived`. В противном случае результатом приведения будет 0. Если указатель `dp` нулевой, условие оператора `if` не выполняется. В этом случае блок директивы `else` осуществляет действия, соответствующие классу `Base`.



Оператор `dynamic_cast` применим и к нулевому указателю; результат — пустой указатель требуемого типа.

Обратите внимание на то, что указатель `dp` определен в условии. При определении переменной в условии приведение и соответствующая проверка осуществляются как единая операция. Кроме того, указатель `dp` недоступен вне оператора `if`. Если приведение потерпит неудачу, то несвязанный указатель не будет доступен для использования в последующем коде, где уже будет забыто успешно ли приведение или нет.



Выполнение оператора `dynamic_cast` в условии гарантирует, что приведение и проверка его результата будут осуществлены в одном выражении.

Приведение `dynamic_cast` для типа ссылки

Приведение `dynamic_cast` для ссылочного типа отличается от такового для типа указателя способом сообщения об ошибке. Поскольку нет такого понятия, как пустая ссылка, для них невозможно использовать ту же стратегию сообщений об ошибке, что и для указателей. Когда приведение к ссылочному типу терпит неудачу, передается исключение `std::bad_cast`, определенное в библиотечном заголовке `typeinfo`.

Предыдущий пример можно переписать так, чтобы использовать ссылки следующим образом:

```
void f(const Base &b)
{
    try {
        const Derived &d = dynamic_cast<const Derived&>(b);
        // использование объекта Derived, на который ссылается b
    } catch (bad_cast) {
        // обработка события неудачи приведения
    }
}
```

Упражнения раздела 19.2.1

Упражнение 19.3. С учетом следующей иерархии классов, где каждый класс определяет открытый стандартный конструктор и виртуальный деструктор:

```
class A { /* ... */ };
class B : public A { /* ... */ };
class C : public B { /* ... */ };
class D : public B, public A { /* ... */ };
```

укажите ошибочные операторы `dynamic_cast` (если таковые имеются).

- (a) `A *pa = new C;`
`B *pb = dynamic_cast< B* >(pa);`
- (b) `B *pb = new B;`
`C *pc = dynamic_cast< C* >(pb);`
- (c) `A *pa = new D;`
`B *pb = dynamic_cast< B* >(pa);`

Упражнение 19.4. Используя классы, определенные в первом упражнении, перепишите следующий код так, чтобы преобразовать выражение `*pa` в тип `C`:

```
if (C *pc = dynamic_cast< C* >(pa))
    // используются члены класса C
} else {
    // используются члены класса A
}
```

Упражнение 19.5. Когда стоит использовать оператор `dynamic_cast` вместо виртуальной функции?

19.2.2. Оператор `typeid`

Второй оператор поддержки RTTI — это оператор `typeid`. Оператор `typeid` позволяет выяснить текущий тип объекта.

Выражение `typeid` имеет форму `typeid(e)`, где `e` — любое выражение или имя типа. Результатом оператора `typeid` является ссылка на константный объект библиотечного типа `type_info` или типа, открыто производного от него. В разделе 19.2.4 (стр. 1037) этот тип рассматривается более подробно. Класс `type_info` определен в заголовке `typeinfo`.

Оператор `typeid` применим к выражениям любого типа. Как обычно, спецификатор `const` верхнего уровня (см. раздел 2.4.3, стр. 101) игнорируется, и если выражение является ссылкой, то оператор `typeid` возвращает тип, на который ссылается ссылка. Но при применении к массиву или функции стандартное преобразование в указатель (см. раздел 4.11.2, стр. 221) не осуществляется. Таким образом, результат выражения `typeid(a)`, где `a` является массивом, описывает тип массива, а не тип указателя.

Когда операнд не имеет типа класса или является классом без виртуальных функций, оператор `typeid` возвращает статический тип операнда. Когда операнд является l-значением типа класса, определяющим по крайней мере одну виртуальную функцию, тип результата вычисляется во время выполнения.

Использование оператора `typeid`

Чаще всего оператор `typeid` используют для сравнения типов двух выражений или для сравнения типа выражения с определенным типом:

```
Derived *dp = new Derived;
Base *bp = dp; // оба указателя указывают на объект Derived
// сравнить типы двух объектов во время выполнения
if (typeid(*bp) == typeid(*dp)) {
    // bp и dp указывают на объекты того же типа
}
// проверить, совпадает ли тип времени выполнения с указанным типом
if (typeid(*bp) == typeid(Derived)) {
    // bp на самом деле указывает на класс Derived
}
```

В первом операторе `if` сравниваются динамические типы объектов, на которые указывают указатели `bp` и `dp`. Если оба указателя указывают на тот же тип, то условие истинно. Точно так же второй оператор `if` истин, если указатель `bp` в настоящее время указывает на объект класса `Derived`.

Обратите внимание: операндами оператора `typeid` являются проверяемые объекты (`*bp`), а не указатели (`bp`).

```
// результат проверки всегда ложный: тип bp - указатель на класс Base
if (typeid(bp) == typeid(Derived)) {
    // код, который никогда не будет выполнен
}
```

Это условие сравнивает тип `Base*` с типом `Derived`. Хотя указатель указывает на объект типа класса, обладающего виртуальными функциями, сам указатель не является объектом типа класса. Тип `Base*` может быть вычислен и вычисляется во время компиляции. Этот тип не совпадает с типом `Derived`, поэтому условие всегда будет ложно, независимо от типа объекта, на который указывает указатель `bp`.



Применение оператора `typeid` к указателю (в отличие от объекта, на который указывает указатель) возвращает статический тип времени компиляции указателя.

Оператор `typeid` требует, чтобы проверка во время выполнения определила, обрабатывается ли выражение. Компилятор обрабатывает выражение, только если у типа есть виртуальные функции. Если у типа нет никаких виртуальных функций, то оператор `typeid` возвращает статический тип выражения; статический тип известен компилятору и без вычисления выражения.

Если динамический тип выражения может отличаться от статического, то выражение следует вычислить (во время выполнения), чтобы определить результирующий тип. Это различие имеет значение при выполнении оператора `typeid(*p)`. Если `p` указывает на тип без виртуальных функций, то указатель `p` не обязан быть допустимым указателем. В противном случае выражение `*p` вычисляется во время выполнения, тогда указатель `p` обязан быть допустимым. Если указатель `p` пуст, то выражение `typeid(*p)` передаст исключение `bad_typeid`.

Упражнения раздела 19.2.2

Упражнение 19.6. Напишите выражение для динамического приведения указателя на тип `Query_base` к указателю на тип `AndQuery` (см. раздел 15.9.1, стр. 802). Проверьте приведение, используя объект класса `AndQuery` и класса другого запроса. Выведите сообщение, подтверждающее работоспособность приведения, и убедитесь, что вывод соответствует ожиданиям.

Упражнение 19.7. Напишите то же приведение, но приведите объект класса `Query_base` к ссылке на тип `AndQuery`. Повторите проверку и удостоверьтесь в правильности работы приведения.

Упражнение 19.8. Напишите выражение `typeid`, чтобы убедиться, указывают ли два указателя на класс `Query_base` на тот же тип. Затем проверьте, не является ли этот тип классом `AndQuery`.

19.2.3. Использование RTTI

В качестве примера случая, когда может пригодиться RTTI, рассмотрим иерархию класса, для которого желательно реализовать оператор равенства (см. раздел 14.3.1, стр. 711). Два объекта равны, если у них тот же тип и то же значение для заданного набора переменных-членов. Каждый производный тип может добавлять собственные данные, которые придется включать в набор проверяемых на равенство.

Казалось бы, эту проблему можно решить, определив набор виртуальных функций, которые проверяют равенство на каждом уровне иерархии. Сделав

оператор равенства виртуальным, можно было бы определить одну функцию, которая работает со ссылкой на базовый класс. Этот оператор мог бы передать свою работу виртуальной функции `equal()`, которая и осуществляла бы все необходимые действия.

К сожалению, виртуальные функции не очень хороши для решения этой задачи. Параметры виртуальной функции должны иметь одинаковые типы и в базовом, и в производных классах (см. раздел 15.3, стр. 765). Если бы пришлось определить виртуальную функцию `equal()`, то ее параметр был бы ссылкой на базовый класс. Если параметр является ссылкой на базовый класс, то функция `equal()` сможет использовать только члены из базового класса. Функция `equal()` никак не могла бы сравнить члены, определенные в производном классе.

Оператор равенства должен возвращать значение `false` при попытке сравнить объекты разных типов. Например, если попытаться сравнивать объект базового класса с объектом производного, оператор `==` должен вернуть значение `false`.

С учетом этого наблюдения можно прийти к выводу, что решить данную проблему можно с использованием RTTI. Определим оператор равенства, параметр которого будет ссылкой на тип базового класса. Оператор равенства будет использовать оператор `typeid` для проверки наличия у операндов одинакового типа. Если тип операндов разный, оператор возвратит значение `false`. В противном случае он возвратит виртуальную функцию `equal()`. Каждый класс определит функцию `equal()` так, чтобы сравнить переменные-члены собственного типа. Эти операторы получают параметр типа `Base&`, но приводят операнд к собственному типу, прежде чем начать сравнение.

Иерархия класса

Чтобы сделать концепцию более конкретной, предположим, что рассматриваемые классы выглядят следующим образом:

```
class Base {
    friend bool operator==(const Base&, const Base&);
public:
    // члены интерфейса для класса Base
protected:
    virtual bool equal(const Base&) const;
    // данные и другие члены реализации класса Base
};
class Derived: public Base {
public:
    // данные и другие члены реализации класса Base
protected:
    bool equal(const Base&) const;
    // данные и другие члены реализации класса Derived
};
```

Оператор равенства, чувствительный к типу

Рассмотрим, как можно было бы определить общий оператор равенства:

```
bool operator==(const Base &lhs, const Base &rhs)
{
    // возвращает false, если типы не совпадают; в противном случае вызов
    // виртуальной функции equal()
    return typeid(lhs) == typeid(rhs) && lhs.equal(rhs);
}
```

Этот оператор возвращает значение `false`, если операнды имеют разный тип. Если они имеют одинаковый тип, оператор делегирует реальную работу по сравнению операндов виртуальной функции `equal()`. Если операнды являются объектами класса `Base`, вызывается функция `Base::equal()`, а если объектами класса `Derived` — то функция `Derived::equal()`.

Виртуальная функция `equal()`

Каждый класс иерархии должен иметь собственную версию функции `equal()`. Начало у функций всех производных классов будет одинаковым: они приводят аргумент к типу собственного класса:

```
bool Derived::equal(const Base &rhs) const
{
    // известно, что типы равны, значит, приведение не передаст
    // исключения
    auto r = dynamic_cast<const Derived*>(rhs);
    // действия по сравнению двух объектов класса Derived и возвращению
    // результата
}
```

Приведение всегда должно быть успешным, ведь оператор равенства вызывает эти функции только после проверки того, что два операнда имеют одинаковый тип. Однако приведение необходимо, чтобы функция могла обращаться к производным членам правого операнда.

Функция `equal()` базового класса

Эта функция гораздо проще других:

```
bool Base::equal(const Base &rhs) const
{
    // действия по сравнению двух объектов класса Base
}
```

Здесь нет никакой необходимости в приведении аргументов перед применением. Оба они, и `*this` и параметр, являются объектами класса `Base`, поэтому все доступные для него функции содержатся в классе объекта.

19.2.4. Класс `type_info`

Точное определение класса `type_info` зависит от компилятора, но стандарт гарантирует, что класс будет определен в заголовке `typeinfo` и предоставлять, по крайней мере, те функции, которые перечислены в табл. 19.1.

Этот класс обладает также открытым виртуальным деструктором, поскольку он предназначен для использования в качестве базового класса. Если компилятор позволяет предоставить дополнительную информацию о типе, для этого следует воспользоваться классом, производным от класса `type_info`.

Таблица 19.1. Функции класса `type_info`

<code>t1 == t2</code>	Возвращает значение <code>true</code> , если оба объекта (<code>t1</code> и <code>t2</code>) имеют тот же тип, и значение <code>false</code> — в противном случае
<code>t1 != t2</code>	Возвращает значение <code>true</code> , если оба объекта (<code>t1</code> и <code>t2</code>) имеют разные типы, и значение <code>false</code> — в противном случае
<code>t.name()</code>	Возвращает символьную строку в стиле C, содержащую отображаемую версию имени типа. Имена типов создаются способом, не зависящим от системы
<code>t1.before(t2)</code>	Возвращает логическое значение (тип <code>bool</code>), указывающее на то, следует ли тип <code>t1</code> прежде типа <code>t2</code> . Порядок следования зависит от компилятора

У класса `type_info` нет стандартного конструктора, а оператор присвоения, конструктор копий и перемещения определены как удаленные (см. раздел 13.1.6, стр. 643). Поэтому нельзя определять, копировать или присваивать объекты типа `type_info`. Единственный способ создания объектов класса `type_info` — это оператор `typeid`.

Функция-член `name()` возвращает символьную строку в стиле C, содержащую имя класса объекта. Значение, используемое для данного типа, зависит от компилятора и не обязательно соответствует имени класса, использованному в программе. Единственное, что гарантирует функция `name()`, — это уникальность возвращаемой ей строки для данного типа.

Рассмотрим пример:

```
int arr[10];
Derived d;
Base *p = &d;
cout << typeid(42).name() << ", "
      << typeid(arr).name() << ", "
      << typeid(Sales_data).name() << ", "
      << typeid(std::string).name() << ", "
      << typeid(p).name() << ", "
      << typeid(*p).name() << endl;
```

При запуске на машине авторов эта программа выводит следующее:

```
i, A10_i, 10Sales_data, Ss, P4Base, 7Derived
```



Класс `type_info` зависит от компилятора. Некоторые компиляторы предоставляют и другие функции-члены, которые возвращают дополнительную информацию о типах, используемых в программе. Чтобы выяснить реальные возможности класса `type_info` для конкретного компилятора, необходимо обратиться к его документации.

Упражнения раздела 19.2.4

Упражнение 19.9. Напишите программу, подобную приведенной в конце этого раздела, для вывода имен, используемых компилятором для общих типов. Если ваш компилятор создает вывод, подобный нашему, напишите функцию, которая преобразует эти строки в более понятную для человека форму.

Упражнение 19.10. С учетом приведенной ниже иерархии классов, в которой каждый класс обладает открытым стандартным конструктором и виртуальным деструктором, укажите, какие имена типов отобразят следующие операторы?

```
class A { /* ... */ };
class B : public A { /* ... */ };
class C : public B { /* ... */ };
```

- (a) `A *pa = new C;`
`cout << typeid(pa).name() << endl;`
- (b) `C cobj;`
`A& ra = cobj;`
`cout << typeid(&ra).name() << endl;`
- (c) `B *px = new B;`
`A& ra = *px;`
`cout << typeid(ra).name() << endl;`

19.3. Перечисления

Перечисления (enumeration) позволяют группировать наборы целочисленных констант. Как и класс, каждое перечисление определяет новый тип. Перечисления — литеральные типы (см. раздел 7.5.6, стр. 385).



В языке C++ есть два вида перечислений: с ограниченной и с не ограниченной областью видимости. *Перечисление с ограниченной областью видимости* (scoped enumeration) вводит новый стандарт. Для определения перечисления с ограниченной областью видимости используются ключевые слова `enum class` (или `enum struct`), сопровождаемые именем

перечисления и разделяемым запятыми списком *перечислителей* (enumerator), заключенным в фигурные скобки. За закрывающей фигурной скобкой следует точка с запятой:

```
enum class open_modes {input, output, append};
```

Здесь определен тип перечисления `open_modes` с тремя перечислителями: `input`, `output` и `append`.

В определении *перечисления с не ограниченной областью видимости* (unscoped enumeration) ключевое слово `class` (или `struct`) отсутствует. Имя перечисления с не ограниченной областью видимости не является обязательным:

```
enum color {red, yellow, green}; // перечисление с не ограниченной
                                // областью видимости
// безымянное перечисление с не ограниченной областью видимости
enum {floatPrec = 6, doublePrec = 10, double_doublePrec = 10};
```

Если перечисление является безымянным, определить объекты его типа можно только в составе определения перечисления. Подобно определению класса, здесь можно предоставить разделяемый запятыми список объявлений между закрывающей фигурной скобкой и точкой с запятой, завершающей определение перечисления (см. раздел 2.6.1 стр. 113).

Перечислители

Имена перечислителей в перечислении с ограниченной областью видимости подчиняются обычным правилам областей видимости и недоступны вне области видимости перечисления. Имена перечислителей в перечислении с не ограниченной областью видимости находятся в той же области видимости, что и само перечисление:

```
enum color {red, yellow, green}; // перечисление с не ограниченной
                                // областью видимости
enum stoplight {red, yellow, green}; // ошибка: переопределение
                                    // перечислителей
enum class peppers {red, yellow, green}; // ok: перечислители скрываются
color eyes = green; // ok: перечислители находятся в области видимости
                    // для перечисления с не ограниченной областью видимости
peppers p = green; // ошибка: перечислители из peppers не находятся в
                  // области видимости
                  // color::находится в области видимости, но имеет
                  // неправильный тип
color hair = color::red; // ok: к перечислителям можно обратиться явно
peppers p2 = peppers::red; // ok: использование red из peppers
```

По умолчанию значения перечислителей начинаются с 0, и значение каждого последующего перечислителя на 1 больше предыдущего. Однако вполне можно предоставить инициализаторы для одного или нескольких перечислителей:

```
enum class intTypes {
    charTyp = 8, shortTyp = 16, intTyp = 16,
    longTyp = 32, long_longTyp = 64
};
```

Как можно заметить на примере перечислителей `intTyp` и `shortTyp`, значение перечислителя не обязано быть уникальным. Без инициализатора значение перечислителя будет на 1 больше, чем у предыдущего.

Перечислители являются константами, и их инициализаторы должны быть константными выражениями (см. раздел 2.4.4, стр. 103). Следовательно, каждый перечислитель сам является константным выражением. Поскольку перечислители — константные выражения, их можно использовать там, где необходимы константные выражения. Например, можно определить переменные `constexpr` типа перечисления:

```
constexpr intTypes charbits = intTypes::charTyp;
```

Точно так же перечисление можно использовать как выражение в операторе `switch`, а значения его перечислителей как метки разделов `case` (см. раздел 5.3.2, стр. 241). По той же причине тип перечисления можно также использовать как параметр значения шаблона (см. раздел 16.1.1, стр. 824) и инициализировать статические переменные-члены типа перечисления в определении класса (см. раздел 7.6, стр. 390).

Подобно классам, перечисления определяют новые типы

Поскольку перечисление имеет имя, можно определять и инициализировать объекты этого типа. Объект перечисления может быть инициализирован или присвоен только одному из своих перечислителей или другому объекту того же типа перечисления:

```
open_modes om = 2;           // ошибка: 2 не имеет типа open_modes
om = open_modes::input;      // ok: input- перечислитель open_modes
```

Объекты или перечислители типа перечисления с не ограниченной областью видимости автоматически преобразовываются в целочисленный тип. В результате они применимы там, где требуется целочисленное значение:

```
int i = color::red;           // ok: перечислитель перечисления с не ограниченной
                               // областью видимости неявно преобразован в тип int
int j = peppers::red;         // ошибка: перечисления с ограниченной областью
                               // видимости неявно не преобразуются
```

Определение размера перечисления

C++ 11 Хотя каждое перечисление определяет уникальный тип, оно представляется одним из встроенных целочисленных типов. По новому стандарту можно указать, что следует использовать тип, заданный за именем перечисления и двоеточием:

```
enum intValues : unsigned long long {
    charTyp = 255, shortTyp = 65535, intTyp = 65535,
    longTyp = 4294967295UL,
    long_longTyp = 18446744073709551615ULL
};
```

Если базовый тип не задан, то по умолчанию перечисления с ограниченной областью видимости имеют базовый тип `int`. Для перечислений с не ограниченной областью видимости типа по умолчанию нет; известно только то, что базовый тип достаточно велик для содержания значения перечислителя. Когда базовый тип определяется (включая неявное определение для перечисления с ограниченной областью видимости), попытка создания перечислителя, значение которого превосходит заданный тип, приведет к ошибке.

Возможность определить базовый тип перечисления позволяет контролировать тип, используемый при разных реализациях компилятора. Это позволяет также гарантировать, что программа, откомпилированная на одной реализации, создаст тот же код при компиляции на другом.

Предварительные объявления для перечислений

**C++
11**

По новому стандарту перечисление можно объявить предварительно. Предварительное объявление перечисления должно определить (неявно или явно) его базовый размер:

```
// предварительное объявление перечисления с не ограниченной областью
// видимости intValues
enum intValues : unsigned long long; // перечисление с не ограниченной
// областью видимости должно определять тип
enum class open_modes; // перечисление с ограниченной областью видимости
// может использовать по умолчанию тип int
```

Поскольку для перечисления с не ограниченной областью видимости нет размера по умолчанию, каждое объявление должно включить его размер. Перечисление с ограниченной областью видимости можно объявить, не определяя размер, тогда размер неявно определяется как `int`.

Подобно любым объявлениям, все объявления и определения того же перечисления должны соответствовать друг другу. В случае перечислений это требование означает, что размер перечисления должен совпадать для всех объявлений и определений. Кроме того, нельзя объявить имя как перечисление с не ограниченной областью видимости в одном контексте, а затем повторно объявить его как перечисление с ограниченной областью видимости:

```
// ошибка: в объявлении и определении должно совпадать, ограничена ли
// область видимости перечисления
enum class intValues;
enum intValues; // ошибка: intValues ранее объявлено как перечисление с
// ограниченной областью видимости
enum intValues : long; // ошибка: intValues ранее объявлено как int
```

Соответствие параметров и перечисления

Поскольку объект типа перечисления может быть инициализирован только другим объектом того же типа перечисления или одним из его перечислителей (см. раздел 19.3, стр. 1039), целое число, значение которого случайно совпадает со значением перечислителя, не может использоваться при вызове функции, ожидающей перечислимый аргумент:

```
// перечисление с не ограниченной областью видимости; базовый тип зависит
// от машины
enum Tokens {INLINE = 128, VIRTUAL = 129};
void ff(Tokens);
void ff(int);
int main() {
    Tokens curTok = INLINE;
    ff(128);      // точно соответствует ff(int)
    ff(INLINE);   // точно соответствует ff(Tokens)
    ff(curTok);   // точно соответствует ff(Tokens)
    return 0;
}
```

Хоть и нельзя передать целочисленное значение параметру перечислимого типа, вполне можно передать объект или перечислитель перечисления с не ограниченной областью видимости параметру целочисленного типа. При этом значение перечислителя преобразуется в тип `int` или больший целочисленный тип. Фактический тип преобразования зависит от базового типа перечисления:

```
void newf(unsigned char);
void newf(int);
unsigned char uc = VIRTUAL;
newf(VIRTUAL); // вызов newf(int)
newf(uc);      // вызов newf(unsigned char)
```

У перечисления `Tokens` только два перечислителя, больший из них имеет значение 129. Это значение может быть представлено типом `unsigned char`, и большинство компиляторов будут использовать для перечисления `Tokens` базовый тип `unsigned char`. Независимо от своего базового типа, объекты и перечислители перечисления `Tokens` преобразуются в тип `int`. Перечислители и значения перечислимого типа не преобразуются в тип `unsigned char`, даже если ему соответствуют значения перечислителей.

19.4. Указатель на член класса

Указатель на член класса (*pointer to member*) — это указатель, способный указывать на нестатический член класса. Обычно указатель указывает на объект, но указатель на член класса идентифицирует только член класса объекта, а не весь объект. Статические члены класса не являются частью конкретного

объекта, поэтому для указания на них не нужен никакой специальный синтаксис. Указатели на статические члены являются обычными указателями.

Тип указателя на член класса объединяет тип класса и тип члена этого класса. Такие указатели инициализируют как указывающие на определенный член класса, не указывая объект, которому принадлежит этот член. При применении указателя на член класса предоставляется объект, член класса которого предстоит использовать.

Для демонстрации работы указателей на члены класса воспользуемся упрощенной версией класса `Screen` из раздела 7.3.1 (стр. 353):

```
class Screen {
public:
    typedef std::string::size_type pos;
    char get_cursor() const { return contents[cursor]; }
    char get() const;
    char get(pos ht, pos wd) const;
private:
    std::string contents;
    pos cursor;
    pos height, width;
};
```

19.4.1. Указатели на переменные-члены

Подобно любым указателям, при объявлении указателя на член класса используется символ `*`, означающий, что объявляемое имя является указателем. В отличие от обычных указателей, указатель на член класса включает также имя класса, содержащего этот член. Следовательно, символу `*` должна предшествовать часть `имяКласса::`, означающая, что определяемый указатель способен указывать на член класса `имяКласса`. Например:

```
// pdata может указывать на член типа string константного (или не
// константного) объекта класса Screen
const string Screen::*pdata;
```

Приведенный выше код объявляет `pdata` “указателем на член класса `Screen`, обладающий типом `const string`”. Переменные-члены константного объекта сами являются константами. Объявление указателя `pdata` как указателя на тип `const string` позволяет использовать его для указания на член любого объекта класса `Screen`, константного или нет. В замен указатель `pdata` применим только для чтения, но не для записи в член класса, на который он указывает.

При инициализации (или присвоении) указателя на член класса следует заявить, на который член он указывает. Например, можно заставить указать `pdata` указывать на переменную-член `contents` неопределенного объекта класса `Screen` следующим образом:

```
pdata = &Screen::contents;
```

Здесь оператор обращения к адресу применяется не к объекту в памяти, а к члену класса `Screen`.

Конечно, по новому стандарту проще объявить указатель на член класса при помощи ключевых слов `auto` или `decltype`:

```
auto pdata = &Screen::contents;
```

Использование указателей на переменные-члены

Важно понять, что при инициализации или присвоении указателя на член класса он еще не указывает на данные. Он идентифицирует определенный член класса, но не содержащий его объект. Объект предоставляется при обращении к значению указателя на член класса.

Подобно операторам доступа к членам (member access operator), `.` и `->`, существуют два оператора доступа к указателю на член класса, `.*` и `->*`, позволяющие предоставить объект и обращаться к значению указателя для доступа к члену этого объекта:

```
Screen myScreen, *pScreen = &myScreen;
// .* обращение к значению pdata для доступа к содержимому члена данного
// объекта класса myScreen
auto s = myScreen.*pdata;
// ->* обращение к значению pdata для доступа к содержимому члена
// объекта, на который указывает pScreen
s = pScreen->*pdata;
```

Концептуально эти операторы выполняют два действия: обращаются к значению указателя на член класса, чтобы получить доступ к необходимому члену; затем, подобно операторам обращения к членам, они обращаются к члену данного объекта непосредственно (`.*`) или через указатель (`->*`).

Функция, возвращающая указатель на переменную-член

К указателям на члены применимы обычные средства управления доступом. Например, член `contents` класса `Screen` является закрытым. В результате указатель `pdata` выше должен использоваться в члене класса `Screen`, его дружественном классе, либо произойдет ошибка.

Поскольку переменные-члены обычно являются закрытыми, как правило, нельзя получать указатель на саму переменную-член. Вместо этого, если такой класс, как `Screen`, желает предоставить доступ к своему члену `contents`, то он определил бы функцию, возвращающую указатель на эту переменную-член:

```
class Screen {
public:
    // data() - статический член, возвращающий указатель на член класса
    static const std::string Screen::data()
    { return &Screen::contents; }
    // другие члены, как прежде
};
```

Здесь в класс `Screen` добавлена статическая функция-член, возвращающая указатель на переменную-член `contents` класса `Screen`. Тип возвращаемого значения этой функции совпадает с типом первоначального указателя `pdata`. Читая тип возвращаемого значения справа налево, можно заметить, что функция `data()` возвращает указатель на член класса `Screen`, имеющий тип `string` и являющийся константой. Тело функции применяет оператор обращения к адресу к переменной-члену `contents`. Таким образом, функция возвращает указатель на переменную-член `contents` класса `Screen`.

Когда происходит вызов функции `data()`, возвращается указатель на член класса:

```
// data() возвращает указатель на член contents класса Screen
const string Screen::pdata = Screen::data();
```

Как и прежде, указатель `pdata` указывает на член класса `Screen`, но не на фактические данные. Чтобы использовать указатель `pdata`, следует связать его с объектом типа `Screen`:

```
// получить содержимое объекта myScreen
auto s = myScreen.*pdata;
```

Упражнения раздела 19.4.1

Упражнение 19.11. В чем разница между обычным указателем на данные и указателем на переменную-член?

Упражнение 19.12. Определите указатель на член класса, способный указывать на член `cursor` класса `Screen`. Получите через этот указатель значение `Screen::cursor`.

Упражнение 19.13. Определите тип, способный представить указатель на член `bookNo` класса `Sales_data`.

19.4.2. Указатели на функции-члены

Вполне можно также определить указатель, способный указывать на функцию-член класса. Подобно указателям на переменные-члены, самый простой способ создания указателя на функцию-член — это использовать ключевое слово `auto` для автоматического вывода типа:

```
// указатель pmf способен указывать на функцию-член класса Screen,
// возвращающую тип char и не получающую никаких аргументов
auto pmf = &Screen::get_cursor;
```

Как и указатель на переменную-член, указатель на функцию-член объявляется с использованием синтаксиса `имяКласса::*`. Подобно любому другому указателю на функцию (см. раздел 6.7, стр. 322), указатель на функцию-член

определяет тип возвращаемого значения и список типов параметров функции, на которую может указывать этот указатель. Если функция-член является константной (см. раздел 7.1.2, стр. 337) или ссылочной (см. раздел 13.6.3, стр. 691), следует также добавить квалификатор `const` или квалификатор ссылки.

Подобно обычным указателям на функцию, если функция-член перегружена, следует явно указать, какая именно функция имеется в виду (см. раздел 6.7, стр. 323). Например, указатель на версию функции `get()` с двумя параметрами можно объявить так:

```
char (Screen::*pmf2)(Screen::pos, Screen::pos) const;
pmf2 = &Screen::get;
```

Круглые скобки вокруг части `Screen::*` в этом объявлении необходимы из за приоритета. Без круглых скобок компилятор воспримет следующий код как (недопустимое) объявление функции:

```
// ошибка: у функции, не являющейся членом класса p, не может быть
// спецификатора const
char Screen::*p(Screen::pos, Screen::pos) const;
```

Это объявление пытается определить обычную функцию по имени `p`, которая возвращает указатель на член класса `Screen` типа `char`. Поскольку объявляется обычная функция, за объявлением не может быть спецификатора `const`.

В отличие от обычных указателей на функцию, нет никакого автоматического преобразования между функцией-членом и указателем на этот член:

```
// pmf указывает на член класса Screen, не получающий аргументов и
// возвращающий тип char
pmf = &Screen::get; // нужно явно использовать оператор обращения к
                   // адресу
pmf = Screen::get;  // ошибка: нет преобразования в указатель для
                   // функций-членов
```

Использование указателя на функцию-член

Как и при использовании указателя на переменную-член, для вызова функции-члена через указатель на член класса используются операторы `.*` и `->.*`:

```
Screen myScreen, *pScreen = &myScreen;
// вызов функции, на которую указывает указатель pmf объекта, на который
// указывает указатель pScreen
char c1 = (pScreen->.*pmf)();
// передает аргументы 0, 0 версии функции get() с двумя параметрами
// объекта myScreen
char c2 = (myScreen.*pmf2)(0, 0);
```

Вызовы `(myScreen->.*pmf)()` и `(pScreen.*pmf2)(0,0)` требуют круглых скобок, поскольку приоритет оператора вызова выше, чем приоритет оператора указателя на член класса.

Без круглых скобок вызов `myScreen.*pmf()` был бы интерпретирован как `myScreen.*(pmf())`.

Этот код требует вызвать функцию `pmf()` и использовать ее возвращаемое значение как операнд оператора указателя на член класса (`.*`). Но `pmf` — не функция, поэтому данный код ошибочен.



Из-за разницы приоритетов операторов вызова объявления указателей на функции-члены и вызовы через такие указатели должны использовать круглые скобки: `(C::*p)(parms)` и `(obj.*p)(args)`.

Использование псевдонимов типов для указателей на члены

Псевдонимы типа или `typedef` (см. разделе 2.5.1, стр. 106) существенно облегчают чтение указателей на члены. Например, следующий код определяет псевдоним типа `Action` как альтернативное имя для типа версии функции `get()` с двумя параметрами:

```
// Action — тип, способный указывать на функцию-член класса Screen,
// возвращающую тип char и получающую два аргумента типа pos
using Action =
char (Screen::*)(Screen::pos, Screen::pos) const;
```

`Action` — это другое имя для типа “указатель на константную функцию-член класса `Screen`, получающую два параметра типа `pos` и возвращающую тип `char`”. Используя этот псевдоним, можно упростить определение указателя на функцию `get()` следующим образом:

```
Action get = &Screen::get; // get указывает на член get() класса Screen
```

Подобно любым другим указателям на функцию, тип указателя на функцию-член можно использовать как тип возвращаемого значения или как тип параметра функции. Подобно любому другому параметру, у параметра указателя на член класса может быть аргумент по умолчанию:

```
// action() получает ссылку на класс Screen и указатель на его
// функцию-член
Screen& action(Screen&, Action = &Screen::get);
```

Функция `action()` получает два параметра, которые являются ссылками на объект класса `Screen`, и указатель на функцию-член класса `Screen`, получающую два параметра типа `pos` и возвращающую тип `char`. Функцию `action()` можно вызвать, передав ей указатель или адрес соответствующей функции-члена класса `Screen`:

```
Screen myScreen;
// эквивалентные вызовы:
action(myScreen); // использует аргумент по умолчанию
action(myScreen, get); // использует предварительно определенную
// переменную get
action(myScreen, &Screen::get); // передает адрес явно
```



Псевдонимы типа облегчают чтение и написание кода, использующего указатели.

Таблицы указателей на функцию-член

Как правило, перед использованием указатели на функции и указатели на функции-члены хранят в таблице функций (см. раздел 14.8.3, стр. 730). Когда у класса есть несколько членов того же типа, такая таблица применяется для выбора одного из набора этих членов. Предположим, что класс `Screen` дополнен несколькими функциями-членами, каждая из которых перемещает курсор в определенном направлении:

```
class Screen {
public:
    // другие члены интерфейса и реализации, как прежде
    Screen& home(); // функции перемещения курсора
    Screen& forward();
    Screen& back();
    Screen& up();
    Screen& down();
};
```

Каждая из этих новых функций не получает никаких параметров и возвращает ссылку на вызвавший ее объект класса `Screen`.

Можно определить функцию `move()`, способную вызвать любую из этих функций и выполнить указанное действие. Для поддержки этой новой функции в класс `Screen` добавлен статический член, являющийся массивом указателей на функции перемещения курсора:

```
class Screen {
public:
    // другие члены интерфейса и реализации, как прежде
    // Action - указатель, который может быть присвоен любой из
    // функций-членов перемещения курсора
    using Action = Screen& (Screen::*)();
    // задать направление перемещения;
    // перечисления описаны в разделе 19.3 на стр. 1038
    enum Directions { HOME, FORWARD, BACK, UP, DOWN };
    Screen& move(Directions);
private:
    static Action Menu[]; // таблица функций
};
```

Массив `Menu` содержит указатели на каждую из функций перемещения курсора. Эти функции будут храниться со смещениями, соответствующими перечислителям перечисления `Directions`. Функция `move()` получает перечислитель и вызывает соответствующую функцию:

```
Screen& Screen::move(Directions cm)
{
    // запустить элемент по индексу cm для объекта this
    return (this->*Menu[cm])(); // Menu[cm] указывает на функцию-член
}
```

Вызов `move()` обрабатывается следующим образом: выбирается элемент массива `Menu` по индексу `cm`. Этот элемент является указателем на функцию-член класса `Screen`. Происходит вызов функции-члена, на которую указывает этот элемент от имени объекта, на который указывает указатель `this`.

Когда происходит вызов функции `move()`, ему передается перечислитель, указывающий направление перемещения курсора:

```
Screen myScreen;
myScreen.move(Screen::HOME); // вызывает myScreen.home
myScreen.move(Screen::DOWN); // вызывает myScreen.down
```

Остается только определить и инициализировать саму таблицу:

```
Screen::Action Screen::Menu[] = { &Screen::home,
                                   &Screen::forward,
                                   &Screen::back,
                                   &Screen::up,
                                   &Screen::down,
                                   };
```

Упражнения раздела 19.4.2

Упражнение 19.14. Корректен ли следующий код? Если да, то что он делает? Если нет, то почему?

```
auto pmf = &Screen::get_cursor;
pmf = &Screen::get;
```

Упражнение 19.15. В чем разница между обычным указателем на функцию и указателем на функцию-член?

Упражнение 19.16. Напишите псевдоним типа, являющийся синонимом для указателя, способного указать на переменную-член `avg_price` класса `Sales_data`.

Упражнение 19.17. Определите псевдоним типа для каждого отдельного типа функции-члена класса `Screen`.

19.4.3. Использование функций-членов как вызываемых объектов

Как уже упоминалось, для вызова через указатель на функцию-член, нужно использовать операторы `.*` и `->*` для связи указателя с определенным объектом. В результате, в отличие от обычных указателей на функцию, указатель на функцию-член класса не является *вызываемым объектом*; эти указатели не поддерживают оператор вызова функции (см. раздел 10.3.2, стр. 497).

Поскольку указатель на член класса не является вызываемым объектом, нельзя непосредственно передать указатель на функцию-член алгоритму. На-

пример, если необходимо найти первую пустую строку в векторе строк, вполне очевидный вызов не сработает:

```
auto fp = &string::empty; // fp указывает на функцию empty()
                          // класса string
// ошибка: для вызова через указатель на член класса следует использовать
// оператор .* или ->*
find_if(svec.begin(), svec.end(), fp);
```

Алгоритм `find_if()` ожидает вызываемый объект, но предоставляется указатель на функцию-член `fp`. Этот вызов не будет откомпилирован, поскольку код в алгоритме `find_if()` выполняет примерно такой оператор:

```
// проверяет применимость данного предиката к текущему элементу,
// возвращает true
if (fp(*it)) // ошибка: для вызова через указатель на член класса следует
// использовать оператор ->*
```

Использование шаблона `function` для создания вызываемого объекта

Один из способов получения вызываемого объекта из указателя на функцию-член подразумевает использование библиотечного шаблона `function` (см. раздел 14.8.3, стр. 730):

```
function<bool (const string&)> fcn = &string::empty;
find_if(svec.begin(), svec.end(), fcn);
```

Здесь шаблону `function` указано, что `empty()` — это функция, которая может быть вызвана со строкой и возвращает значение типа `bool`. Обычно объект, для которого выполняется функция-член, передается неявному параметру `this`. Когда шаблон `function` используется при создании вызываемого объекта для функции-члена, следует преобразовать код так, чтобы сделать этот неявный параметр явным.

Когда объект шаблона `function` содержит указатель на функцию-член, класс `function` знает, что для вызова следует использовать соответствующий оператор указателя на член класса. Таким образом, можно предположить, что у функции `find_if()` будет код наподобие следующего:

```
// если it является итератором в функции find_if(), то *it - объект
// в заданном диапазоне
if (fcn(*it)) // fcn - имя вызываемого объекта в функции find_if()
```

Его и выполнит шаблон класса `function`, используя соответствующий оператор указателя на член класса. Класс `function` преобразует этот вызов в такой код:

```
// если it является итератором в функции find_if(), то *it - объект
// в заданном диапазоне
if ((*it).*p)() // p - указатель на функцию-член в функции fcn
```


При определении объекта шаблона `function` следует указать тип функции, сигнатура которой определяет представляемые вызываемые объекты. Когда вызываемый объект является функцией-членом, первый параметр сигнатуры должен представить (обычно неявный) объект, для которого будет выполнена функция-член. Передаваемая шаблону `function` сигнатура должна определять, будет ли объект передан как указатель или как ссылка.

При определении вызываемого объекта `fcn()` было известно, что нужно вызвать функцию `find_if()` для последовательности строковых объектов. Следовательно, от шаблона `function` требовалось создать вызываемый объект, получающий объекты класса `string`. Если бы вектор содержал указатели на тип `string`, от шаблона `function` требовалось бы ожидать указатель:

```
vector<string*> pvec;
function<bool (const string*)> fp = &string::empty;
// fp получает указатель на string и использует оператор ->* для вызова
// функции empty()
find_if(pvec.begin(), pvec.end(), fp);
```

Использование шаблона `mem_fn` для создания вызываемого объекта

**C++
11** Чтобы использовать шаблон `function`, следует предоставить сигнатуру вызова члена, который предстоит вызвать. Но можно позволить компилятору вывести тип функции-члена при использовании другого библиотечного средства, шаблона `mem_fn`, определенного, как и шаблон `function`, в заголовке `functional`. Как и шаблон `function`, шаблон `mem_fn` создает вызываемый объект из указателя на член класса. В отличие от шаблона `function`, шаблон `mem_fn` выведет тип вызываемого объекта из типа указателя на член класса:

```
find_if(svec.begin(), svec.end(), mem_fn(&string::empty));
```

Здесь шаблон `mem_fn(&string::empty)` создает вызываемый объект, получающий строковый аргумент и возвращающий логическое значение.

Вызываемый объект, созданный шаблоном `mem_fn`, может быть вызван для объекта или указателя:

```
auto f = mem_fn(&string::empty); // f получает string или string*
f(*svec.begin()); // ok: передача объекта string; f использует .* для
                  // вызова empty()
f(&svec[0]);       // ok: передача указателя на string; f использует ->
                  // для вызова empty()
```

Фактически шаблон `mem_fn` можно считать как *будто* создающим вызываемый объект с перегруженным оператором вызова функции — один получает тип `string*`, а другой — `string&`.

Использование функции `bind()` для создания вызываемого объекта

Для создания вызываемого объекта из функции-члена можно также использовать функцию `bind()` (см. раздел 10.3.4, стр. 508):

```
// связать каждую строку из диапазона с неявным первым аргументом empty()
auto it = find_if(svec.begin(), svec.end(),
    bind(&string::empty, _1));
```

Подобно шаблону `function`, при использовании функции `bind()` следует сделать явным обычно неявный параметр функции-члена, представляющий объект, с которым будет работать функция-член. Подобно шаблону `mem_fn`, первый аргумент вызываемого объекта, создаваемого функцией `bind()`, может быть либо указателем, либо ссылкой на тип `string`:

```
auto f = bind(&string::empty, _1);
f(*svec.begin()); // ok: аргумент - строка f, использует .* для вызова
                  // функции empty()
f(&svec[0]);       // ok: аргумент - указатель на строку f использует .->
                  // для вызова функции empty()
```

Упражнения раздела 19.4.3

Упражнение 19.18. Напишите функцию, использующую алгоритм `count_if()` для подсчета количества пустых строк в заданном векторе.

Упражнение 19.19. Напишите функцию, получающую вектор `vector<Sales_data>` и находящую первый элемент, средняя цена которого превосходит заданное значение.

19.5. Вложенные классы

Класс, определяемый в другом классе, называется *вложенным классом* (nested class) или *вложенным типом* (nested type). Вложенные классы обычно используются для классов реализации, как, например, класс `QueryResult` из приложения текстового запроса (см. раздел 12.3, стр. 617).

Имя вложенного класса видимо в области видимости содержащего его класса, но не вне ее. Имя вложенного класса не будет входить в конфликт с тем же именем, объявленным в другой области видимости.

Вложенный класс может содержать члены тех же видов, что и невложенный класс. Подобно любому другому классу, вложенный класс контролирует доступ к своим членам при помощи спецификаторов доступа. Содержащий класс не имеет никаких специальных прав доступа к членам вложенного класса, а вложенный класс не имеет привилегий в доступе к членам содержащего его класса.

В содержащем классе вложенный класс представляет собой член, типом которого является класс. Подобно любому другому члену, содержащий класс задает уровень доступа к этому типу. Вложенный класс, определенный в разделе `public` содержащего класса, может быть использован везде. Вложенный класс, определенный в разделе `protected`, доступен только содержащему классу, его производным и дружественным классам. Вложенный класс, определенный в разделе `private`, доступен лишь для членов содержащего класса и классов, дружественных для него.

Объявление вложенного класса

Класс `TextQuery` из раздела 12.3.2 (стр. 620) определял сопутствующий класс `QueryResult`. Класс `QueryResult` жестко связан с классом `TextQuery`. Класс `QueryResult` имело бы смысл использовать и для других целей, а не только для результатов операции запроса к объекту класса `TextQuery`. Для отражения этой жесткой связи сделаем класс `QueryResult` членом класса `TextQuery`.

```
class TextQuery {
public:
    class QueryResult; // вложенный класс будет определен позже
    // другие члены, как в разделе 12.3.2 (стр. 620)
};
```

В первоначальный класс `TextQuery` необходимо внести только одно изменение — объявить о намерении определить класс `QueryResult` как вложенный. Поскольку класс `QueryResult` будет типом-членом (см. раздел 7.3.4, стр. 368), его следует объявить прежде, чем использовать. В частности, класс `QueryResult` следует объявить прежде, чем использовать его как тип возвращаемого значения функции-члена `query()`. Остальные члены первоначального класса неизменны.

Определение вложенного класса вне содержащего класса

В классе `TextQuery` класс `QueryResult` объявлен, но не определен. Подобно функциям-членам, вложенные классы следует объявить в классе, но определен он может быть в или вне класса.

При определении вложенного класса вне его содержащего класса следует квалифицировать имя вложенного класса именем его содержащего класса:

```
// определение класса QueryResult как члена класса TextQuery
class TextQuery::QueryResult {
    // в области видимости класса не нужно квалифицировать имя
    // параметров QueryResult
    friend std::ostream&
        print(std::ostream&, const QueryResult&);
public:
    // не нужно определять QueryResult::line_no; вложенный класс способен
    // использовать член своего содержащего класса без необходимости
```

```
// квалифицировать его имя
QueryResult(std::string,
            std::shared_ptr<std::set<line_no>>,
            std::shared_ptr<std::vector<std::string>>);
// другие члены, как в разделе 12.3.2 (стр. 620)
};
```

Единственное изменение, внесенное в первоначальный класс, заключается в том, что в классе `QueryResult` больше не определяется переменная-член `line_no`. Члены класса `QueryResult` могут обращаться к этому имени непосредственно в классе `TextQuery`, таким образом, нет никакой необходимости определять его снова.



Пока не встретится фактическое определение вложенного класса, расположенное вне тела класса, этот класс является незавершенным типом (см. раздел 7.3.3, стр. 362).

Определение членов вложенного класса

В этой версии конструктор `QueryResult()` не определяется в теле класса. Чтобы определить конструктор, следует указать, что класс `QueryResult` вложен в пределы класса `TextQuery`. Для этого имя вложенного класса квалифицируют именем содержащего его класса:

```
// определение члена класса по имени QueryResult для класса по
// имени QueryResult, вложенного в класс TextQuery
TextQuery::QueryResult::QueryResult(string s,
                                   shared_ptr<set<line_no>> p,
                                   shared_ptr<vector<string>> f):
    sought(s), lines(p), file(f) { }
```

Читая имя функции справа налево, можно заметить, что это определение конструктора для класса `QueryResult`, который вложен в пределы класса `TextQuery`. Сам код только сохраняет данные аргументов в переменных-членах и не делает больше ничего.

Определение статических членов вложенных классов

Если бы класс `QueryResult` объявлял статический член, его определение находилось бы вне области видимости класса `TextQuery`. Например, статический член класса `QueryResult` был бы определен как-то так:

```
// определение статического члена типа int класса QueryResult
// вложенного в класс TextQuery
int TextQuery::QueryResult::static_mem = 1024;
```

Поиск имен в области видимости вложенного класса

Во вложенном классе выполняются обычные правила поиска имен (см. раздел 7.4.1, стр. 367). Конечно, поскольку вложенный класс — это вложенная область видимости, для поиска у него есть дополнительные области видимости

в содержащем классе. Такое вложение областей видимости объясняет, почему переменная-член `line_no` не определялась во вложенной версии класса `QueryResult`. Первоначальный класс `QueryResult` определял этот член для того, чтобы его собственные члены могли избежать необходимости записи `TextQuery::line_no`. После вложения определения класса результатов в класс `TextQuery` такое определение типа больше не нужно. Вложенный класс `QueryResult` может обратиться к переменной `line_no` без указания, что она определена в классе `TextQuery`.

Как уже упоминалось, вложенный класс — это тип-член содержащего его класса. Члены содержащего класса могут использовать имена вложенного класса таким же образом, как и любой другой тип-член. Поскольку класс `QueryResult` вложен в класс `TextQuery`, функция-член `query()` класса `TextQuery` может обращаться к имени `QueryResult` непосредственно:

```
// тип возвращаемого значения должен указать, что класс QueryResult
// теперь вложенный
TextQuery::QueryResult
TextQuery::query(const string &sought) const
{
    // если искомое значение не найдено, вернуть указатель на этот
    // набор
    static shared_ptr<set<line_no>> nodata(new set<line_no>);
    // во избежания добавления слов к wm использовать поиск, а не
    // индексирование!
    auto loc = wm.find(sought);
    if (loc == wm.end())
        return QueryResult(sought, nodata, file); // не найдено
    else
        return QueryResult(sought, loc->second, file);
}
```

Как обычно, тип возвращаемого значения не находится в области видимости класса (см. раздел 7.4, стр. 366), поэтому сразу было обращено внимание на то, что функция возвращает значение типа `TextQuery::QueryResult`. Но в теле функции к типу `QueryResult` можно обращаться непосредственно, как это сделано в операторах `return`.

Вложенные и содержащие классы независимы

Несмотря на то что вложенный класс определяется в пределах содержащего его класса, важно понимать, что никакой связи между объектами содержащего класса и объектами его вложенного класса (классов) нет. Объект вложенного типа только содержит члены, определенные во вложенном типе. Точно так же у объекта содержащего класса есть только те члены, которые определяются содержащим классом. Он не содержит переменные-члены любых вложенных классов.

Конкретней, второй оператор `return` в функции-члене `TextQuery::query()` использует переменные-члены объекта класса `TextQuery`, для которого была выполнена функция `query()`, инициализирующая объект класса `QueryResult`:

```
return QueryResult(sought, loc->second, file);
```

Эти члены используются для создания возвращаемого объекта класса `QueryResult`, поскольку он не содержит члены содержащего его класса.

Упражнения раздела 19.5

Упражнение 19.20. Вложите собственный класс `QueryResult` в класс `TextQuery` и повторно запустите написанную в разделе 12.3.2 (стр. 624) программу, использующую класс `TextQuery`.

19.6. Класс объединения, экономящий место

Класс *объединения* (`union`) — это специальный вид класса. У него может быть несколько переменных-членов, но в любой момент времени значение может быть только у одного из членов. Когда присваивается значение одному из членов класса объединения, все остальные члены становятся неопределенными. Объем хранилища, резервируемого для объединения, достаточен для содержания наибольшей переменной-члена. Подобно любому классу, класс объединения определяет новый тип.

Некоторые, но не все средства класса объединения применяются одинаково. У класса объединения не может быть члена, являющегося ссылкой, но у него могут быть члены большинства других типов, включая, согласно новому стандарту, типы классов с конструкторами или деструкторами. Объединение может использовать спецификаторы доступа, чтобы сделать члены открытыми, закрытыми, или защищенными. По умолчанию, как и у структуры, члены объединения являются открытыми.

Класс объединения может определять функции-члены, включая конструкторы и деструкторы. Но объединения не могут происходить от другого класса и не могут быть использованы как базовый класс. В результате у объединения не может быть виртуальных функций.

Определение объединения

Объединения позволяют создать набор взаимоисключающих значений, которые могут иметь разные типы. Предположим, например, что существует процесс, в ходе которого обрабатываются различные виды числовых или символьных данных. Для хранения этих значений можно было бы использовать следующее объединение.

```
// объект типа Token способен содержать один член, имеющий любой из
// следующих типов
union Token {
// члены по умолчанию открыты
    char cval;
    int ival;
    double dval;
};
```

Определение объединения начинается с ключевого слова `union`, за которым следует имя объединения (не обязательно) и набор его членов, заключенный в фигурные скобки. Этот код определяет объединение по имени `Token`, способное содержать значение типа `char`, `int` или `double`.

Использование объединения

Имя объединения — это имя типа. Подобно встроенным типам, по умолчанию объединения не инициализированы. Объединение можно явно инициализировать таким же образом, как и агрегатные классы (см. раздел 7.5.5, стр. 384), — при помощи инициализаторов, заключенных в фигурные скобки:

```
Token first_token = {'a'}; // инициализирует член cval
Token last_token; // не инициализированный объект Token
Token *pt = new Token; // указатель на не инициализированный
// объект Token
```

Если инициализатор есть, он используется для инициализации первого члена. Следовательно, инициализация объединения `first_token` присваивает значение его члену `cval`.

К членам объекта типа объединения обращаются при помощи обычных операторов доступа к члену:

```
last_token.cval = 'z';
pt->ival = 42;
```

Присвоение значения переменной-члену объекта объединения делает другие его переменные-члены неопределенными. В результате при использовании объединения следует всегда знать, какое именно значение в настоящее время хранится в нем. В зависимости от типов членов возвращение или присвоение хранимого в объединении значения при помощи неправильной переменной-члена может привести к аварийному отказу или неправильному поведению программы.

Анонимные объединения

Анонимное объединение (`anonymous union`) — это безымянное объединение, не содержащее объявлений между закрывающей фигурной скобкой, завершающей его тело, и точкой с запятой, завершающей определение объединения (см. раздел 2.6.1, стр. 113). При определении анонимного объединения

компилятор автоматически создает безымянный объект только что определенного типа объединения:

```
union { // анонимное объединение
    char cval;
    int ival;
    double dval;
}; // определяет безымянный объект, к членам которого можно обращаться
// непосредственно
cval = 'с'; // присваивает новое значение безымянному, анонимному объекту
// объединения
ival = 42; // теперь этот объект содержит значение 42
```

Члены анонимного объединения непосредственно доступны в той области видимости, где определено анонимное объединение.



У анонимного объединения не может быть закрытых или защищенных членов, кроме того, оно не может определять функции-члены.

Объединения с членами типа класса

**C++
11**

По прежним стандартам языка C++ у объединений не могло быть членов типа класса, которые определяли бы собственные конструкторы или функции-члены управления копированием. По новому стандарту это ограничение снято. Однако объединения с членами, способными определять собственные конструкторы и (или) функции-члены управления копированием, куда сложнее в применении, чем объединения только с членами встроенного типа.

Если у объединения есть члены только встроенного типа, для изменения содержащегося в нем значения можно использовать обычное присвоение. С объединениями, у которых есть члены нетривиальных типов, все не так просто. При присвоении или замене значения члена объединения типа класса следует создать или удалить этот член соответственно: при присвоении объединению значения типа класса следует запустить конструктор для типа данного элемента, а при замене — запустить его деструктор.

Если у объединения есть члены только встроенного типа, компилятор сам синтезирует почленные версии стандартного конструктора и функций-членов управления копированием. Но для объединений, у которых есть член типа класса, определяющего собственный стандартный конструктор или функция-член управления копированием, это не так. Если тип члена объединения определяет одну из этих функций-членов, компилятор синтезирует соответствующий член объединения как удаленный (см. раздел 13.1.6, стр. 644).

Например, класс `string` определяет все пять функций-членов управления копированием, а также стандартный конструктор. Если объединение будет

содержать строку и не определит ее собственный стандартный конструктор или одну из функций-членов управления копированием, то компилятор синтезирует эту недостающую функцию как удаленную. Если у класса будет член типа объединения, у которого есть удаленная функция-член управления копированием, то соответствующая функция (функции) управления копированием самого класса также будет удалена.

Использование класса для управления членами объединения

Из-за сложностей создания и удаления членов типа класса такие объединения обычно встраивают в другой класс. Таким образом, класс получает возможность управлять состоянием при передаче из и в элемент типа класса. В качестве примера добавим в объединение член класса `string`. Определим объединение как анонимное и сделаем его членом класса `Token`. Класс `Token` будет управлять членами объединения.

Для отслеживания вида значения хранимого объединением обычно определяют отдельный объект, *дискриминант* (*discriminant*). Дискриминант позволяет различать значения, которые может содержать объединение. Для синхронизации объединения и его дискриминанта сделаем дискриминант также членом класса `Token`. Для отслеживания состояния члена объединения класс определит член типа перечисления (см. раздел 19.3, стр. 1038).

Единственными определяемыми классом функциями будут стандартный конструктор, функции-члены управления копированием и ряд операторов присвоения, способных присваивать значение одного из типов объединения члену другого:

```
class Token {
public:
    // функции управления копированием необходимы потому, что у класса
    // есть объединение с членом типа string
    // определение конструктора перемещения и оператора присваивания при
    // перемещении остается в качестве самостоятельного упражнения
    Token(): tok(INT), ival{0} { }
    Token(const Token &t): tok(t.tok) { copyUnion(t); }
    Token &operator=(const Token&);
    // если объединение содержит строку, ее придется удалять;
    // см. раздел 19.1.2 (стр. 1028)
    ~Token() { if (tok == STR) sval.~string(); }
    // операторы присвоения для установки разных членов объединения
    Token &operator=(const std::string&);
    Token &operator=(char);
    Token &operator=(int);
    Token &operator=(double);
private:
    enum {INT, CHAR, DBL, STR} tok; // дискриминант
    union {                          // анонимное объединение
        char sval;
        int ival;
    };
};
```

```

    double dval;
    std::string sval;
}; // у каждого объекта класса Token есть безымянный член типа этого
    // безымянного объединения
// проверить дискриминант и скопировать член объединения, как надо
void copyUnion(const Token&);
};

```

Класс определяет вложенное, безымянное перечисление с не ограниченной областью видимости (см. раздел 19.3, стр. 1038), используемое как тип члена `tok`. Член `tok` определен после закрывающей фигурной скобки и перед точкой с запятой, завершающей определение перечисления, которое определяет `tok`, как имеющий тип этого безымянного перечисления (см. раздел 2.6.1, стр. 113).

Член `tok` будет использован как дискриминант. Когда объединение содержит значение типа `int`, член `tok` будет содержать значение `INT`; если объединение содержит значение типа `string`, то член `tok` содержит значение `STR` и т.д.

Стандартный конструктор инициализирует дискриминант и член объединения как содержащие значение 0 типа `int`.

Поскольку объединение содержит член, класс которого обладает деструктором, следует определить собственный деструктор, чтобы (условно) удалять член типа `string`. В отличие от обычных членов типа класса, члены типа класса, являющиеся частью объединения, не удаляются автоматически. У деструктора нет никакого способа узнать, значение какого типа хранит объединение. Таким образом, он не может знать, какой из членов следует удалить.

Поэтому деструктор проверяет, не содержит ли удаляемый объект строку. Если это так, то деструктор явно вызывает деструктор класса `string` (см. раздел 19.1.2, стр. 1028) для освобождения используемой памяти. Если объединение содержит значение любого из встроенных типов, то деструктор не делает ничего.

Управление дискриминанта и удаление строки

Операторы присвоения устанавливают значение переменной `tok` и присваивают соответствующий член объединения. Подобно деструктору, эти функции-члены должны условно удалять строку, прежде чем присваивать новое значение объединению:

```

Token &Token::operator=(int i)
{
    if (tok == STR) sval.~string(); // если это строка, освободить ее
    ival = i;                       // присвоить соответствующий член
    tok = INT;                      // обновить дискриминант
    return *this;
}

```

Если текущим значением объединения является строка, ее следует освободить прежде, чем присвоить объединению новое значение. Для этого используется деструктор класса `string`.

Как только член типа `string` освобождается, предоставленное значение присваивается члену, тип которого соответствует типу параметра оператора. В данном случае параметр имеет тип `int`, поэтому он присваивается `ival`. Затем обновляется дискриминант и осуществляется выход.

Операторы присвоения для типов `double` и `char` ведут себя, как и версия для типа `int`, их определение остается в качестве самостоятельного упражнения. Версия для типа `string` отличается от других, поскольку она должна управлять переходом от типа `string` и к нему:

```
Token &Token::operator=(const std::string &s)
{
    if (tok == STR) // если строка уже содержится, просто присвоить новую
        sval = s;
    else
        new(&sval) string(s); // в противном случае создать строку
    tok = STR; // обновить дискриминант
    return *this;
}
```

В данном случае, если объединение уже содержит строку, можно использовать обычный оператор присвоения класса `string`, чтобы предоставить новое значение существующей строке. В противном случае не будет никакого объекта класса `string` для вызова его оператора присвоения. Вместо этого придется создать строку в памяти, которая содержит объединение. Для создания строки в области, где располагается `sval`, используется размещающий оператор `new` (см. раздел 19.1.2, стр. 1028). Строка инициализируется копией строкового параметра, затем обновляется дискриминант и осуществляется выход.

Управление членами объединения, требующее управления копированием

Подобно специфическим для типа операторам присвоения, конструктор копий и операторы присвоения должны проверять дискриминант, чтобы знать, как копировать переданное значение. Для выполнения этих действий определим функцию-член `copyUnion()`.

Когда происходит вызов функции `copyUnion()` из конструктора копий, член объединения будет инициализирован значением по умолчанию, означая, что будет инициализирован первый член объединения. Поскольку строка не является первым элементом, вполне очевидно, что объединение содержит не строку. Оператор присвоения должен учитывать возможность того, что объединение уже содержит строку. Отработаем этот случай непосредственно в операторе присвоения. Таким образом, если параметр функции `copyUnion()` содержит строку, она должна создать собственную строку:

```

void Token::copyUnion(const Token &t)
{
    switch (t.tok) {
        case Token::INT: ival = t.ival; break;
        case Token::CHAR: cval = t.cval; break;
        case Token::DBL: dval = t.dval; break;
        // для копирования строки создать ее, используя размещающий
        // оператор new; см. раздел 19.1.2 (стр. 1028)
        case Token::STR: new(&sval) string(t.sval); break;
    }
}

```

Для проверки дискриминанта эта функция использует оператор `switch` (см. раздел 5.3.2, стр. 241). Значения встроенных типов просто присваиваются соответствующему члену; если копируемый член имеет тип `string`, он создается.

Оператор присвоения должен отработать три возможности для члена типа `string`: левый и правый операнды являются строками; ни один из операндов не является строкой; один, но не оба операнда являются строкой:

```

Token &Token::operator=(const Token &t)
{
    // если этот объект содержит строку, а t нет, прежнюю строку следует
    // освободить
    if (tok == STR && t.tok != STR) sval.~string();
    if (tok == STR && t.tok == STR)
        sval = t.sval; // нет необходимости создавать новую строку
    else
        copyUnion(t); // создать строку, если t.tok содержит STR
    tok = t.tok;
    return *this;
}

```

Если объединение в левом операнде содержит строку, а объединение в правом — нет, то сначала следует освободить прежнюю старую строку, прежде чем присваивать новое значение члену объединения. Если оба объединения содержат строку, для копирования можно использовать обычный оператор присвоения класса `string`. В противном случае происходит вызов функции `copyUnion()`, осуществляющей присвоение. В функции `copyUnion()`, если правый операнд — строка, создается новая строка в члене объединения левого операнда. Если ни один из операндов не будет строкой, то достаточно обычного присвоения.

Упражнения раздела 19.6

Упражнение 19.21. Напишите собственную версию класса `Token`.

Упражнение 19.22. Добавьте в класс `Token` член типа `Sales_data`.

Упражнение 19.23. Добавьте в класс `Token` конструктор перемещения и присвоения.

Упражнение 19.24. Объясните, что происходит при присвоении объекта класса `Token` самому себе.

Упражнение 19.25. Напишите операторы присвоения, получающие значения каждого типа в объединении.

19.7. Локальные классы

Класс, определенный в теле функции, называют *локальным классом* (local class). Локальный класс определяет тип, видимый только в той области видимости, в которой он определен. В отличие от вложенных классов, члены локального класса жестко ограничены.



Все члены локального класса, включая функции, должны быть полностью определены в теле класса. В результате локальные классы гораздо менее полезны, чем вложенные.

На практике требование полностью определять члены в самом классе, существенно ограничивает сложность, а следовательно, и возможности функций-членов локального класса. Функции локальных классов редко имеют размер, превышающий несколько строк кода. Более длинный код функций труднее прочитать и понять.

Кроме того, в локальном классе нельзя объявлять статические переменные-члены, поскольку нет никакого способа определить их.

Локальные классы не могут использовать переменные из области видимости функции

Локальный класс может обращаться далеко не ко всем именам из окружающей области видимости. Он может обращаться только к именам типов, статических переменных (см. раздел 6.1.1, стр. 271) и перечислений, определенных в окружающей локальной области видимости. Локальный класс не может использовать обычные локальные переменные той функции, в которой определен класс:

```
int a, val;
void foo(int val)
{
    static int si;
    enum Loc { a = 1024, b };
    // Bar локальна для foo
    struct Bar {
        Loc locVal; // ok: используется локальное имя типа
        int barVal;
        void fooBar(Loc l = a) // ok: аргумент по умолчанию Loc::a
        {
            barVal = val;    // ошибка: val локален для foo
        }
    };
}
```

```

        barVal = ::val; // ok: используется глобальный объект
        barVal = si; // ok: используется статический локальный объект
        locVal = b;    // ok: используется перечислитель
    }
};
// ...
}

```

К локальным классам применимы обычные правила доступа

Содержащая функция не имеет никаких специальных прав доступа к закрытым членам локального класса. Безусловно, локальный класс вполне может сделать содержащую функцию дружественной. Как правило, локальный класс определяет свои члены как открытые. Та часть программы, которая может обращаться к локальному классу, весьма ограничена. Локальный класс сосредоточен (инкапсулирован) в своей локальной области видимости. Дальнейшая инкапсуляция, подразумевающая сокрытие информации, безусловно, является излишней.

Поиск имен в локальном классе

Поиск имен в теле локального класса осуществляется таким же образом, как и у остальных классов. Имена, используемые в объявлениях членов класса, должны быть объявлены в области видимости до их применения. Имена, используемые в определениях членов, могут располагаться в любой части области видимости локального класса. Поиск имен, не найденных среди членов класса, осуществляется сначала в содержащей локальной области видимости, а затем вне области видимости, заключающей саму функцию.

Вложенные локальные классы

Вполне возможно вложить класс в локальный класс. В данном случае определение вложенного класса может располагаться вне тела локального класса. Однако вложенный класс следует определить в той же локальной области видимости, в которой определен локальный класс:

```

void foo()
{
    class Bar {
    public:
        // ...
        class Nested; // объявление класса Nested
    };
    // определение класса Nested
    class Bar::Nested {
        // ...
    };
}

```

Как обычно, при определении члена вне класса следует указать область видимости имени. Следовательно, определение `Bar::Nested` означает класс `Nested`, определенный в пределах класса `Bar`.

Класс, вложенный в локальный класс, сам является локальным классом, со всеми соответствующими ограничениями. Все члены вложенного класса должны быть определены в теле самого вложенного класса.

19.8. Возможности, снижающие переносимость

Для поддержки низкоуровневого программирования язык C++ определяет набор средств, применение которых снижает переносимость приложений. *Непереносимое* (nonportable) средство специфично для определенных машин. Используя такие средства программы зачастую требуют переделки кода при переносе с одной машины на другую. Одной из причин невозможности переноса является тот факт, что размеры арифметических типов на разных машинах разные (см. раздел 2.1.1, стр. 62).

В этом разделе рассматриваются два дополнительных средства, снижающих переносимость, унаследованных языком C++ от языка C: речь идет о битовых полях и спецификаторе `volatile`. Также будут рассмотрены директивы компоновки, которые тоже снижают переносимость.

19.8.1. Битовые поля

Класс может определить (нестатическую) переменную-член как *битовое поле* (bit-field). Битовое поле хранит определенное количество битов. Обычно битовые поля используются при необходимости передать двоичные данные другой программе или аппаратному устройству.



Расположение в памяти битовых полей зависит от конкретной машины.

У битового поля должны быть целочисленный тип или тип перечисления (см. раздел 19.3, стр. 1038). Для битового поля обычно используют беззнаковый тип, поскольку поведение битового поля знакового типа зависит от реализации. Чтобы объявить член класса битовым полем, после его имени располагают двоеточие и константное выражение, указывающее количество битов:

```
typedef unsigned int Bit;
class File {
    Bit mode: 2;           // mode имеет 2 бита
    Bit modified: 1;       // modified имеет 1 бит
    Bit prot_owner: 3;      // prot_owner имеет 3 бита
    Bit prot_group: 3;      // prot_group имеет 3 бита
    Bit prot_world: 3;      // prot_world имеет 3 бита
    // функции и переменные-члены класса File
};
```

```

public:
    // режимы файла определены как восьмеричные
    // литералы; см. р. 2.1.3 стр. 70
    enum modes { READ = 01, WRITE = 02, EXECUTE = 03 };
    File &open(modes);
    void close();
    void write();
    bool isRead() const;
    void setWrite();
}

```

Битовое поле `mode` имеет размер в два бита, битовое поле `modified` — только один, а другие — по три бита. Битовые поля, определенные в последовательном порядке в теле класса, если это возможно, упаковываются в смежных битах того же целого числа. Таким образом достигается уплотнение хранилища. Например, пять битовых полей в приведенном выше объявлении будут сохранены в одной переменной типа `unsigned int`, ассоциированной с первым битовым полем `mode`. Способ упаковки битов в целое число зависит от машины.

К битовому полю не может быть применен оператор обращения к адресу (&), поэтому не может быть никаких указателей на битовые поля классов.



Для битовых полей обычно лучше подходит беззнаковый тип. Поведение битовых полей, хранимых в переменной знакового типа, определяет конкретная реализация.

Использование битовых полей

К битовым полям обращаются так же, как и к другим переменным-членам класса:

```

void File::write()
{
    modified = 1;
    // ...
}
void File::close()
{
    if (modified)
        // ... сохранить содержимое
}

```

Для манипулирования битовыми полями с несколькими битами обычно используют встроенные побитовые операторы (см. раздел 4.8, стр. 210):

```

File &File::open(File::modes m)
{
    mode |= READ; // установить бит READ по умолчанию
    // другая обработка
    if (m & WRITE) // если открыто для чтения и записи
        // процесс открытия файла в режиме чтения/записи
    return *this;
}

```


Классы, определяющие члены битовых полей, обычно определяют также набор встраиваемых функций-членов для проверки и установки значений битовых полей:

```
inline bool File::isRead() const { return mode & READ; }
inline void File::setWrite() { mode |= WRITE; }
```

19.8.2. Спецификатор `volatile`



Смысл спецификатора `volatile` полностью зависит от конкретной машины и может быть выяснен только в документации компилятора. При переносе на новые машины или компиляторы программы, использующие спецификатор `volatile`, обычно приходится переделывать.

Программы, которым приходится работать непосредственно с аппаратными средствами, зачастую имеют элементы данных, значением которых управляют процессы, не контролируемые самой программой. Например, программа могла бы содержать переменную, значение которой изменяет системный таймер. Такой объект должен быть объявлен со спецификатором `volatile`, тогда его значение может быть изменено способами, не контролируемыми или не обнаруживаемыми компилятором. Ключевое слово `volatile` — это приказ компилятору не выполнять оптимизацию для таких объектов.

Спецификатор `volatile` используется аналогично спецификатору `const`, т.е. как дополнительный модификатор типа:

```
volatile int display_register; // значение int может измениться
volatile Task *curr_task;     // curr_task указывает на объект volatile
volatile int iax[max_size];   // каждый элемент в iax volatile
volatile Screen bitmapBuf;    // каждый член bitmapBuf volatile
```

Между спецификаторами типа `const` и `volatile` нет никакой взаимосвязи. Тип может быть и `const`, и `volatile`, тогда у него есть оба качества.

Точно так же класс может определить константные функции-члены, а может и *асинхронно-изменяемые* (`volatile`). Только асинхронно-изменяемые функции-члены могут быть вызваны асинхронно-изменяемым (`volatile`) объектом.

Взаимодействие указателей со спецификатором `const` описано в разделе 2.4.2 (стр. 99). Аналогичное взаимодействие существует между указателями и спецификатором `volatile`. Можно объявлять асинхронно-изменяемые указатели на объекты, указатели на асинхронно-изменяемые объекты и асинхронно-изменяемые указатели на асинхронно-изменяемые объекты.

```
volatile int v; // v - асинхронно-изменяемый объект типа int
int *volatile vip; // vip - асинхронно-изменяемый указатель на тип int
volatile int *ivp; // ivp - указатель на асинхронно-изменяемый тип int
// vip - асинхронно-изменяемый указатель на асинхронно-изменяемый
```

```
// объект типа int
volatile int *volatile vivp;
int *ip = &v; // ошибка: нужен указатель на volatile
*ivp = &v;    // ok: ivp - указатель на volatile
vivp = &v;    // ok: vivp - volatile указатель на volatile
```

Подобно константам, адрес асинхронно-изменяемого объекта можно присвоить (или скопировать указатель на асинхронно-изменяемый тип) только асинхронно-изменяемому указателю. При инициализации ссылки на асинхронно-изменяемый объект следует использовать только асинхронно-изменяемые ссылки.

Синтезируемые функции управления копированием не применимы к асинхронно-изменяемым объектам

Между константными и асинхронно-изменяемыми объектами есть одно важное различие: для инициализации и присвоения асинхронно-изменяемых объектов не применимы синтезируемые версии операторов присвоения, копирования и перемещения. Синтезируемые функции-члены управления копированием получают параметры, типами которых являются константные ссылки на класс. Однако асинхронно-изменяемый объект не может быть передан при помощи обычной или константной ссылки.

Если класс должен обеспечить копирование, перемещение или присвоение асинхронно-изменяемых объектов в (или из) асинхронно-изменяемый операнд, в нем следует определить его собственные версии операторов копирования и перемещения. Например, объявив параметры как ссылки `const` и `volatile`, можно обеспечить копирование или присвоение из любого вида типа `Foo`:

```
class Foo {
public:
    Foo(const volatile Foo&); // копирование из объекта volatile
    // присвоение объекта volatile обычному объекту
    Foo& operator=(volatile const Foo&);
    // присвоение объекта volatile объекту volatile
    Foo& operator=(volatile const Foo&) volatile;
    // остальная часть класса Foo
};
```

Хотя для объектов `volatile` вполне можно определить функции копирования и присвоения, возникает вполне резонный вопрос: имеет ли смысл копировать объект `volatile`? Ответ зависит от причины использования такого объекта в конкретной программе.

19.8.3. Директивы компоновки: `extern "C"`

Иногда в программах C++ необходимо применять функции, написанные на другом языке программирования. Как правило, это язык C. Подобно любому имени, имя функции, написанной на другом языке, следует объявить. Это объяв-

ление должно указать тип возвращаемого значения и список параметров. Компилятор проверяет обращения к внешним функциям на другом языке точно так же, как и обращения к обычным функциям языка C++. Однако для вызова функций, написанных на других языках, компилятор обычно вынужден создавать иной код. Чтобы указать язык для функций, написанных на языке, отличном от C++, используются *директивы компоновки* (linkage directive).



Комбинация кода C++ с кодом, написанным на любом другом языке, включая язык C, требует доступа к компилятору этого языка, совместимому с вашим компилятором C++.

Объявление функций, написанных на языке, отличном от C++

Директива компоновки может существовать в двух формах: одиночной и составной. Директивы компоновки не могут располагаться в определении класса или функции. Некоторые директивы компоновки должны присутствовать в каждом объявлении функции.

В качестве примера рассмотрим некоторые из функций языка C, объявленные в заголовке `stdlib.h`:

```
// гипотетические директивы компоновки, которые могли бы присутствовать в
// заголовке C++ <cstring>
// одиночная директива компоновки
extern "C" size_t strlen(const char *);
// составная директива компоновки
extern "C" {
    int strcmp(const char*, const char*);
    char *strcat(char*, const char*);
}
```

Первая форма состоит из ключевого слова `extern`, сопровождаемого строковым литералом и “обычным” объявлением функции.

Строковый литерал указывает язык, на котором написана функция. Используемый компилятор обязан поддерживать директивы компоновки для языка C. Компилятор может поддерживать директивы компоновки и для других языков, например `extern "Ada"`, `extern "FORTRAN"` и т.д.

Директивы компоновки и заголовки

Та же директива компоновки может быть применена к нескольким функциям одновременно. Для этого их объявления заключают в фигурные скобки после директивы компоновки. Эти фигурные скобки служат для группировки объявлений, к которым применяется директива компоновки. Эти фигурные скобки игнорируются, а имена функций, объявленных в их пределах, видимы, как будто функции были объявлены вне фигурных скобок.

Составная форма объявления применима ко всему файлу заголовка. Например, заголовок `cstring` языка C++ может выглядеть следующим образом.

```
// составная директива компоновки
extern "C" {
#include <string.h> // функции языка C, манипулирующие строками в стиле C
}
```

Когда директива `#include` заключена в фигурные скобки составной директивы компоновки, все объявления обычных функций в файле заголовка будут восприняты как написанные на языке, указанном в директиве компоновки. Директивы компоновки допускают вложенность, т.е. если заголовок содержит функцию с директивой компоновки, на данную функцию это не повлияет.



Функции, унаследованные языком C++ от языка C, могут быть определены как функции языка C, но это не является обязательным условием для каждой реализации языка C++.

Указатели на функции, объявленные в директиве **extern "C"**

Язык, на котором написана функция, является частью ее типа. Чтобы объявить указатель на функцию, написанную на другом языке программирования, следует использовать директиву компоновки. Кроме того, указатели на функции, написанные на других языках, следует объявлять с той же директивой компоновки, что и у самой функции:

```
// pf указывает на функцию C, возвращающую void и получающую int
extern "C" void (*pf)(int);
```

Когда указатель `pf` используется для вызова функции, созданный при компиляции код подразумевает, что происходит обращение к функции C.

Тип указателя на функцию C не совпадает с типом указателя на функцию C++. Указатель на функцию C не может быть инициализирован (или присвоен) значением указателя на функцию C++ (и наоборот). Как и при любом другом несовпадении типов, попытка присвоения указателя с другой директивой компоновки приведет к ошибке:

```
void (*pf1)(int); // указатель на функцию C++
extern "C" void (*pf2)(int); // указатель на функцию C
pf1 = pf2; // ошибка: pf1 и pf2 имеют разные типы
```



Некоторые компиляторы C++ могут допускать присвоение, приведенное выше, хотя, строго говоря, оно некорректно.

Директивы компоновки применимы ко всем объявлениям

Директива компоновки, использованная для функции, применяется также и к любым указателям на нее, используемым как тип возвращаемого значения или параметр.

```
// f1() - функция C, ее параметр также является указателем на функцию C
extern "C" void f1(void(*) (int));
```

Это объявление свидетельствует о том, что `f1()` является функцией языка С, которая не возвращает никаких значений. Она имеет один параметр в виде указателя на функцию, которая ничего не возвращает и получает один параметр типа `int`. Эта директива компоновки применяется как к самой функции `f1()`, так и к указателю на нее. Когда происходит вызов функции `f1()`, ей необходимо передать имя функции С или указатель на нее.

Поскольку директива компоновки применяется ко всем функциям в объявлении, для передачи функции С++ указателя на функцию С необходимо использовать определение типа (см. раздел 2.5.1, стр. 106):

```
// FC - указатель на функцию С
extern "C" typedef void FC(int);
// f2 - функция С++, параметром которой является указатель на функцию С
void f2(FC *);
```

Экспорт функций, созданных на языке С++, в другой язык

Используя директиву компоновки в определении функции, написанной на языке С++, эту функцию можно сделать доступной для программы, написанной на другом языке.

```
// функция calc() может быть вызвана из программы на языке С
extern "C" double calc(double dparm) { /* ... */ }
```

Код, создаваемый компилятором для этой функции, будет соответствовать указанному языку.

Следует заметить, что типы параметров и возвращаемого значения в функциях для разных языков зачастую ограничены. Например, почти наверняка нельзя написать функцию, которая передает объекты нетривиального класса С++ программе на языке С. Программа С не будет знать о конструкторах, деструкторах или других специфических для класса операциях.

ПОДДЕРЖКА ПРЕПРОЦЕССОРА ПРИ КОМПОНОВКЕ НА ЯЗЫКЕ С

Чтобы позволить компилировать тот же файл исходного кода на языке С или С++, при компиляции на языке С++ препроцессор автоматически определяет имя `__cplusplus` (два символа подчеркивания). Используя эту переменную, при компиляции на С++ можно условно включить код, компилируемый только на С++:

```
#ifdef __cplusplus
// ok: компилируется только в С++
extern "C"
#endif
int strcmp(const char*, const char*);
```

Перегруженные функции и директивы компоновки

Взаимодействие директив компоновки и перегрузки функций зависит от конкретного языка. Если язык поддерживает перегрузку функций, то компилятор, обрабатывая директивы компоновки для того языка, вероятней всего, выполнит ее.

Язык С не поддерживает перегрузку функций, поэтому нет ничего удивительного в том, что директива компоновки языка С может быть определена только для одной из функций в наборе перегруженных функций:

```
// ошибка: в директиве extern "C" указаны две одноименные функции
extern "C" void print(const char*);
extern "C" void print(int);
```

Если одна из функций в наборе перегруженных функций является функцией языка С, все остальные функции должны быть функциями С++:

```
class SmallInt { /* ... */ };
class BigNum { /* ... */ };
// функция С может быть вызвана из программ С и С ++
// версия функции С++, перегружающая предыдущую функцию, может быть
// вызвана только из программ на языке С++
extern "C" double calc(double);
extern SmallInt calc(const SmallInt&);
extern BigNum calc(const BigNum&);
```

Версия функции `calc()` для языка С может быть вызвана как из программ на языке С, так и из программ на языке С++. Дополнительные функции с параметрами типа класса могут быть вызваны только из программ на языке С++, причем порядок объявления не имеет значения.

Упражнения раздела 19.8.3

Упражнение 19.26. Объясните эти объявления и укажите, допустимы ли они:

```
extern "C" int compute(int *, int);
extern "C" double compute(double *, double);
```

Резюме

Язык С++ предоставляет несколько специализированных средств, предназначенных для решения ряда специфических проблем.

Некоторым приложениям требуется взять под свой контроль распределение памяти. Это можно сделать, определив собственные версии (в классе или глобально) библиотечных функций `operator new()` и `operator delete()`. Если приложение определяет собственные версии этих функций, выражения `new` и `delete` будут использовать соответствующую версию, определенную приложением.

Некоторым программам необходимо непосредственно выяснять динамический тип объекта во время выполнения. Идентификация типов времени выполнения (Run-Time Type Identification — RTTI) предоставляет поддержку этого вида программирования на уровне языка. RTTI применима только к тем классам, которые обладают виртуальными функциями; информация о типах без виртуальных функций также доступна, но она соответствует статическому типу.

При определении указателя на член класса в состав его типа должен также входить тот класс, на член которого указывает указатель. Указатель на член класса может быть связан с членом любого объекта того же класса. При обращении к значению указателя на член класса необходимо указать объект, о члене которого идет речь.

В языке C++ определено несколько дополнительных составных типов.

- Вложенные классы, которые определены в области видимости другого класса. Такие классы зачастую применяют для реализации содержащего их класса.
- Объединения — это специальный вид класса, объект которого может содержать только простые переменные-члены. В любой момент времени объект такого типа может содержать значение только в одной из его переменных-членов. Как правило, объединения входят в состав другого класса.
- Локальные классы представляют собой очень простые классы, определенные локально в функции. Все члены локального класса должны быть определены в его теле. Для локального класса недопустимы статические переменные-члены.

Язык C++ предоставляет также несколько средств, ухудшающих переносимость программ. Сюда относятся битовые поля, спецификатор `volatile`, упрощающий взаимодействие с аппаратными средствами, и директивы компоновки, упрощающие взаимодействие с программами, написанными на других языках.

Термины

Анонимное объединение (anonymous union). Безымянное объединение, которое не применимо для создания объекта. Члены анонимного объединения являются членами окружающей области видимости. Такие объединения не могут иметь ни функций-членов, ни закрытых или защищенных членов.

Битовое поле (bit-field). Целочисленный член класса, определяющий количество резервируемых для него битов. Битовые поля, определенные в классе последовательно, могут быть упакованы в обычное целочисленное значение.

Вложенный класс (nested class). Класс, определенный в другом классе. Вложенный класс определен в окружающей области видимости: имена вложенных классов должны быть уникальны в области видимости того класса, в котором они определены, но могут повторяться в областях видимости вне содержащего класса. Доступ к вложенному классу извне содержащего класса предполагает применение оператора области видимости, позволяющего указать область (области) видимости, в которую вложен класс.

Вложенный тип (nested type). Синоним вложенного класса.

Директива компоновки (linkage directive). Механизм, позволяющий вызвать в программе на языке C++ функции, написанные на другом языке. Вызов функций C должны поддерживать все компиляторы языка C++. Поддержка других языков зависит от конкретного компилятора.

Идентификация типов времени выполнения (run-time type identification). Языковые и библиотечные средства, позволяющие выяснить динамический тип ссылки или указателя во время выполнения. Операторы `RTTI`, `typeid` и `dynamic_cast`, обеспечиваяют возвращение динамического типа только для ссылок и указателей на классы с виртуальными функциями. Будучи примененными к другим типам, они возвращают статический тип ссылки или указателя.

Локальный класс (local class). Класс, определенный в функции. Локальный класс видим только в той функции, в которой он определен. Все его члены должны быть определены в теле класса. Он не может иметь статических членов. Локальные члены класса не могут обращаться к локальным переменным, определенным в содержащей функции. Однако они могут использовать имена типов, статические переменные и перечисления, определенные в содержащей функции.

Непереносимый (nonportable). Специфические для конкретных машин средства, которые могут потребовать изменений при переносе программы на другую машину или компилятор.

Объединение (union). Подобный классу составной тип, в котором может быть определено несколько переменных-членов, однако значение в каждый момент времени может иметь только один из них. Объединения могут иметь функции-члены, включая конструкторы и деструкторы, но они не могут быть использованы в качестве базового класса. По новому стандарту у объединений могут быть члены-типы, определяющие собственные функции-члены управления копированием. Такие объединения получают удаленные функции управления копированием, если они не определяют соответствующие функции управления копированием.

Оператор delete. Библиотечная функция, освобождающая динамическую память без контроля типов, зарезервированную оператором `new`. Библиотечный оператор `delete[]` освобождает память, задействованную массивом, который был зарезервирован оператором `new[]`.

Оператор dynamic_cast. Осуществляет приведение типа базового класса к типу производного с проверкой. В базовом классе должна быть определена по крайней мере одна виртуальная функция. Оператор проверяет динамический тип объекта, с которым связана ссылка или указатель. Приведение осуществляется только тогда, когда тип объекта совпадает с типом приведения или является типом, производным от него.

В противном случае возвращается нулевой указатель (при приведении указателя) или исключение (при приведении ссылки).

Оператор typeid. Унарный оператор, получающий выражение и возвращающий ссылку на объект библиотечного типа `type_info`, описывающего тип полученного выражения. Когда выражение является объектом класса, имеющего виртуальные функции, оператор возвращает динамический тип. Если типом является ссылка, указатель или другой тип, в котором не определены виртуальные функции, будет возвращен его статический тип. Выражение не вычисляется.

Перечисление (enumeration). Тип, группирующий набор именованных целочисленных констант.

Перечисление с не ограниченной областью видимости (unscoped enumeration). Перечисление, перечислители которого доступны в окружающей области видимости.

Перечисление с ограниченной областью видимости (scoped enumeration). Перечисление нового вида, в котором перечислитель не доступен непосредственно в окружающей области видимости.

Перечислитель (enumerator). Именованный член перечисления. Каждый перечислитель инициализируется константным целочисленным значением. Перечислители могут быть использованы там, где необходимы целочисленные константные выражения.

Размещающий оператор new (placement new). Форма оператора `new`, создающая объект в указанной области памяти. Память он не резервирует, а область, предназначенную для объекта, указывает получаемый аргумент. Представляет собой низкоуровневый аналог функции-члена `construct()` класса `allocator`.

Спецификатор volatile. Спецификатор типа, указывающий компилятору на то, что значение переменной данного типа может быть изменено извне программы. Это запрещает компилятору осуществлять некоторые виды оптимизации кода.

Тип type_info. Библиотечный тип, возвращаемый оператором `typeid`. Класс `type_info` жестко зависит от конкретной машины, однако любая библиотека должна определять класс `type_info` как содержащий функцию-член `name()`, возвращающую символьную строку, представляющую имя типа. Объекты класса `type_info` не могут быть скопированы, перемещены или присвоены.

Указатель на член класса (pointer to member). Инкапсулирует тип класса, а также тип элемента, на который он указывает. Определение указателя на член класса должно содержать имя класса, а также тип элемента (элементов), на который он может указывать.

```
T C::*pmem = &C::{member};
```

Это выражение определяет указатель `pmem`, который способен указывать на члены класса по имени `C`, которые имеют тип `T`, и инициализирует его адресом члена класса `C` по имени `member`. Перед обращением к значению такого указателя он должен быть предварительно связан с объектом или указателем класса `C`.

```
classobj.*pmem;  
classptr->*pmem;
```

Обращение к члену `member` объекта `classobj` или указателя `classptr`.

Функция `free()`. Низкоуровневая функция освобождения памяти, определенная в заголовке `cstdlib`. Функция `free()` может использоваться для освобождения *только* той памяти, которая зарезервирована функцией `malloc()`.

Функция `malloc()`. Низкоуровневая функция резервирования памяти, определенная в заголовке `cstdlib`. Зарезервированную функцией `malloc()` память следует освобождать функцией `free()`.

Шаблон `mem_fn`. Библиотечный шаблон класса, создающий вызываемый объект из переданного указателя на функцию-член.